

**Name of Department:- Computer Science and Engineering**

|    |                |                |               |                                                                              |
|----|----------------|----------------|---------------|------------------------------------------------------------------------------|
| 1. | Subject Code:  | <b>TCS 101</b> | Course Title: | <b>FUNDAMENTALS OF<br/>COMPUTERS AND<br/>INTRODUCTION TO<br/>PROGRAMMING</b> |
| 2. | Contact Hours: | L: <b>3</b>    | T: <b>-</b>   | P: <b>-</b>                                                                  |
| 3. | Semester:      | <b>I</b>       |               |                                                                              |

4. Pre-requisite: Basic Knowledge of Mathematics

5. Course Outcomes: After completion of the course students will be able to

- Learn the concepts of IT and understand the fundamentals of basic building blocks of computer science.
- Understand basic data types and syntax of C programming.
- Propose solution to problem by using tools like algorithm and flowcharts.
- Analyze and select best possible solution for decision-based problems using decision making skills and develop the aptitude to solve iterative problems using different types of looping statements.
- Apply and implement the concept arrays for providing solution to homogenous collection of data types.
- Implement complex problem as a collection of sub problems by applying modularization in applications using functions.

6. Detailed Syllabus

| UNIT      | CONTENTS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Contact Hrs |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Unit - I  | Generation of computers, Computer system memory hierarchy, Input/Output, RAM/ROM, Software & Hardware, Understand bit, byte, KB, MB, GB and their relations to each other, Operating System overview, Computer Networks Overview, Algorithms and Flow Charts – Examples of Flow charts for loops and conditional statements.                                                                                                                                                                                                                                                                                                                                                                                                          | 8           |
| Unit - II | First C program - Hello world, How to open a command prompt on Windows or Linux. How to read and print on screen printf(), scanf(), getchar(), putchar() Variables and Data types - Variables, Identifiers, data types and sizes, type conversions, difference between declaration and definition of a variable, Constants<br>Life of a C program (Preprocessing, Compilation, Assembly, Linking, Loading, Execution), Compiling from the command line, Macros, Operators – equality and assignment, Compound assignment operators, Increment and decrement operators, Performance comparison between pre and post increment/decrement operators, bitwise operators, Logical Operators, comma operator, precedence and associativity. | 10          |

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                            |           |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Unit III<br>– | <b>Conditional statements:</b> if statement, if-else statement, ternary statement or ternary operator, nested if-else statement, switch statement, Difference between performance of if else and switch, Advantages of if else and switch over each other<br><b>Loops:</b> 'for' loops, 'while' loops, 'do while' loops, entry control and exit control, break and continue, nested loops                                                  | 8         |
| Unit – IV     | <b>Arrays:</b> Single-dimensional arrays, initializing arrays, computing address of an element in array, character arrays, segmentation fault, bound checking, Searching and Sorting.                                                                                                                                                                                                                                                      | 7         |
| Unit V<br>–   | <b>Functions:</b> Function prototype, function return type, signature of a function, function arguments, call by value, Function call stack, Recursion v/s Iteration, passing arrays to functions,<br><b>Storage classes:</b> Automatic, Static, Register, External, Static and Dynamic linking implementation, C program memory (show different areas of C program memory and where different type of variables are stored), scope rules. |           |
| <b>Total</b>  |                                                                                                                                                                                                                                                                                                                                                                                                                                            | <b>43</b> |

#### Text Books:

- Peter Prinz, Tony Crawford, "C in a Nutshell", 1st Edition, Oreilly Publishers, 2011.
- Peter Norton, "Introduction to computers", 6th Edition, TMH, 2009.

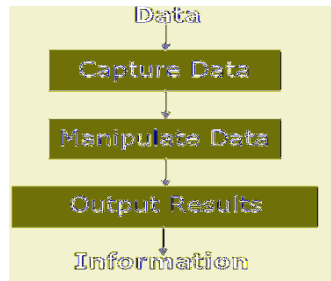
#### Reference Books:

- Steve Oualline, "Practical C programming", 3rd Edition, Orielly Publishers, 2011.
- Brian W Kernighan, Dennis M Ritchie, "The C Programming Language", 2nd Edition, Prentice Hall, 1988. R3. Herbert Schildt, "C: The Complete Reference", 4th Edition. TMH, 2000.
- E. Balagurusamy, "Programming in ANSI C", 6th Edition, McGraw Hill 2015
- Yashwant Kanetkar, "Let Us C", 8th Edition, BPB Publication 2007

## UNIT 1 PART-A

### INTRODUCTION TO COMPUTERS (Additional)

Definition of Computer-The word computer comes from the word “compute”, which means, “to calculate. A computer is an electronic device that process data, converting it into information that is useful to people. A computer is also called a data processor because it can store, process, and retrieve data whenever desired



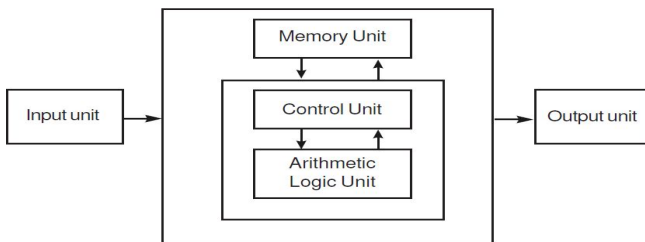
The activity of processing data using a computer is called data processing

### DATA VS. INFORMATION (Additional)

Data is raw, unorganized facts that need to be processed. Data can be something simple and seemingly random and useless until it is organized. Each student's test score is one piece of data.

When data is processed, organized, structured or presented in a given context so as to make it useful, it is called Information. The class' average score or the school's average score is the information that can be concluded from the given data.

### COMPUTER ORGANIZATION- BLOCK DIAGRAM OF COMPUTER (Additional)



All general-purpose computers require the following hardware components:

- 🖨 **Input device** : Usually a [keyboard](#) and [mouse](#), the input device is the conduit through which data and instructions enter a computer.
- 🖨 **Memory** : Enables a computer to [store](#), at least temporarily, data and programs.
- 🖨 **Mass storage device**: Allows a computer to permanently retain large amounts of data. Common mass storage devices include [disk drives](#) and [tape drives](#).
- 🖨 **Output device** : A [display screen](#), [printer](#), or other device that lets you see what the computer has accomplished.
- 🖨 **Central processing unit (CPU)**: The heart of the computer, this is the component that actually executes instructions. The ALU and the CU of a computer system are jointly known as the central processing unit (CPU).
  - **Control Unit (CU)**: The process of input, output, processing and storage is performed under the supervision of a unit called 'Control Unit'. It decides when to start receiving data, when to stop it, where to store data, etc. It takes care of step-by-step processing of all operations inside the computer.
  - **Arithmetic Logic Unit (ALU)**: The major operations performed by the ALU are addition, subtraction, multiplication, division, logic and comparison.

### GENERATION OF COMPUTER (Syllabus)

(Ref: [wikieducator.org](http://wikieducator.org))

As the time passed, the device of more suitable and reliable machine was need which could perform our work more quickly. During this time, in the year 1946, the first successful electronic computer called ENIAC was developed and it was the starting point of the current generation of computer

#### **FIRST GENERATION (1940-1956)**

ENIAC was the world first successful electronic computer which was develops by the two scientists namely J. P. Eckert and U. W. Mauchly. It was the beginning of first generation computer. The full form of ENIAC is

“Electronic Numeric Integrated And Calculator” ENIAC was a very huge and big computer and its weight was 30 tones. It could store only limited or small amount of information. Initially in the first generation computer the concept of vacuum tubes was used. A vacuum tube was such an electronic component which had very less work efficiency and so it could not work properly and it required a large cooling system.

### **SECOND GENERATION (1956-1963)**

As the development moved further, the second generation computers knocked the door. In this generation, transistors were used as the electronic component instead of vacuum tubes. A transistor is much smaller in the size than that of a vacuum tube. As the size of electronic components decreased from vacuum tube of transistor, the size of computer also decreased and it became much smaller than that of earlier computer.

### **THIRD GENERATION (1964-1971)**

The third generation computers were invented in the year 1964. In this generation of computer, IC (Integrated circuits) was used as the electronic component for computers. The development of IC gave birth to a new field of microelectronics. The main advantage of IC is not only its small size but its superior performance and reliability than the previous circuits. It was first developed by T.S Kilby. This generation of computer has huge storage capacity and higher calculating speed.

### **FOURTH GENERATION (1971-Present)**

This is the generation where we are working today. The computers which we see around us belong to the fourth generation computers. ‘Micro processor’ is the main concept behind this generation of computer. A microprocessor is a single chip (L.S.I circuit), which is used in a computer for any arithmetical or logical functions to be performed in any program. The honor of developing microprocessor goes to Ted Hoff of U.S.A. He developed first micro-processor, the Intel 4004, as he was working for Intel Corporation, U.S.A with the use of microprocessor in the fourth generation computers, the size of computer become very fast and efficient.

### **FIFTH GENERATION: ARTIFICIAL INTELLIGENCE (present and beyond)**

Fifth generation computing devices, based on artificial intelligence, are still in development, though there are some applications, such as voice recognition, that are being used today. The use of parallel processing and superconductors is helping to make artificial intelligence a reality. Quantum computation and molecular and nanotechnology will radically change the face of computers in years to come. The goal of fifth-generation computing is to develop devices that respond to natural language input and are capable of learning and self-organization.

## **COMPUTER MEMORY HIERARCHY: INTERNAL REGISTER, CACHE, RAM, HARD DISK, MAGNETIC TAPE (Syllabus)**

Computer memory can be classified in the below given hierarchy:

**1) Internal register:** Internal register in a CPU is used for holding variables and temporary results. Internal registers have a very small storage; however they can be accessed instantly. Accessing data from the internal register is the fastest way to access memory.

**2) Cache:** Cache is used by the CPU for memory which is being accessed over and over again. Instead of pulling it every time from the main memory, it is put in cache for fast access. It is also a smaller memory, however, larger than internal register.

Cache is further classified to L1, L2 and L3:

a) L1 cache: It is accessed without any delay.

b) L2 cache: It takes more clock cycles to access than L1 cache.

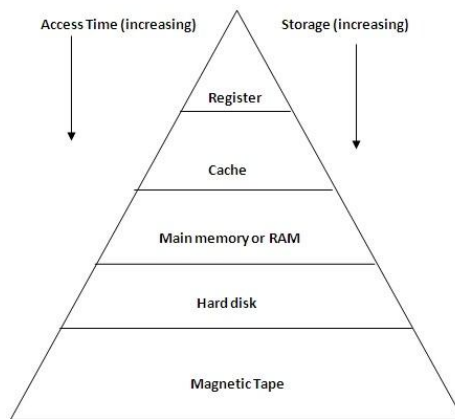
c) L3 cache: It takes more clock cycles to access than L2 cache.

**3) Main memory or RAM (Random Access Memory):** It is a type of the computer memory and is a hardware component. It can be increased provided the operating system can handle it. Typical PCs these days use 8 GB of RAM. It is accessed slowly as compared to cache.

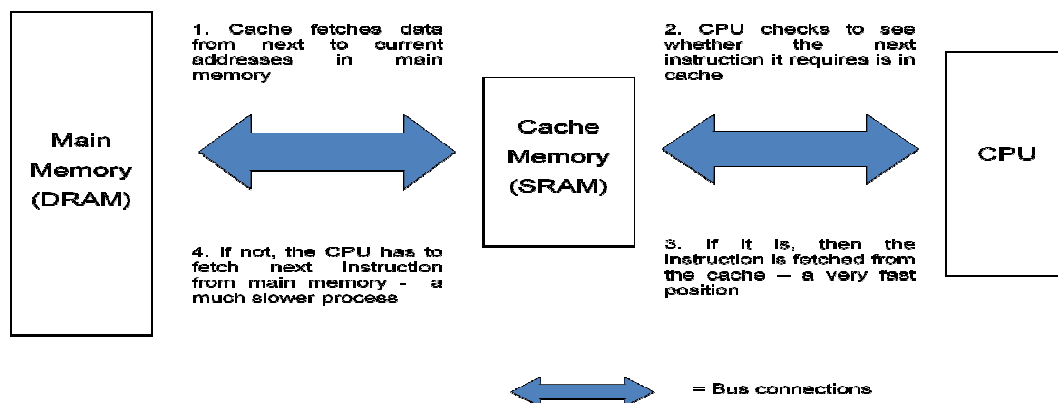
**4) Hard disk:** A hard disk is a hardware component in a computer. Data is kept permanently in this memory. Memory from hard disk is not directly accessed by the CPU, hence it is slower. As compared with RAM, hard disk is cheaper per bit.

**5) Magnetic tape:** Magnetic tape memory is usually used for backing up large data. When the system needs to access a tape, it is first mounted to access the data. When the data is accessed, it is then unmounted. The memory access time is slower in magnetic tape and it usually takes few minutes to access a tape.

Below given figure shows the hierarchy of computer memory:



## The operation of cache memory



(Ref. - <http://www.moreprocess.com>)

## RAM & ROM (Syllabus)

### Memory

Main memory consists of a number of storage locations, each of which is identified by a **unique address**.

The ability of the CPU to identify each location is known as its **addressability**.

Each location stores a **word** i.e. the number of bits that can be processed by the CPU in a single operation.

**Word length** may be typically 16, 24, 32 or as many as 64 bits.

A large word length **improves system performance**, though may be less efficient on occasions when the full word length is not used.

### Types of main memory

There are two types of main memory, **Random Access Memory (RAM)** and **Read Only Memory (ROM)**

#### Random Access Memory (RAM)

- ✓ holds its data as long as the computer is switched on
- ✓ All data in RAM is lost when the computer is switched off
- ✓ Described as being **volatile**
- ✓ It is **direct access** as it can be both written to or read from in any order

Its purpose is to temporarily hold programs and data for processing. In modern computers it also holds the **operating system**

### Types of RAM

#### 1. Dynamic Random Access Memory (DRAM)

- Contents are constantly refreshed 1000 times per second
- Access time 60 – 70 nanoseconds
- Dynamic memory is built with capacitors.
  - A stored charge on the capacitor represents a logical 1.
  - No charge represents a logic 0.
- However, capacitors lose their charge after a few milliseconds. The memory requires constant refreshing to recharge the capacitors. (That's what's "dynamic" about it.)

Dynamic RAMs tend to be physically smaller than static RAMs.

- A single bit of data can be stored with just one capacitor and one transistor, while static RAM cells typically require 4-6 transistors.
- This means dynamic RAM is cheaper and denser—more bits can be stored in the same physical area.

## 2. Synchronous Dynamic Random Access Memory (SDRAM)

- Quicker than DRAM

Access time less than 60 nanoseconds

## 3. Direct Rambus Dynamic Random Access Memory (DRDRAM)

- New type of RAM architecture
- Access time 20 times faster than DRAM
- More expensive

## 4. Static Random Access Memory (SRAM)

- Doesn't need refreshing
- Retains contents as long as power applied to the chip
- Access time around 10 nanoseconds
- Used for **cache** memory

Static RAM (SRAM) has the advantage of being faster than DRAM, although the disadvantage is that it is more expensive. SRAM is static in the sense that it doesn't require constant electrical refreshes; however, it still requires constant current to maintain the voltage differentials. SRAM generally requires less power than DRAM, although its power requirements vary depending on clock speed. At higher clock speeds, it can use as much power as DRAM; however, at more moderate speeds, it requires only a fraction of what DRAM uses. When idle, SRAM power requirements are low.

## 5. Cache memory

- Small amount of memory typically 256 or 512 kilobytes
- Temporary store for often used instructions
- **Level 1** cache is built within the CPU (internal)
- **Level 2** cache may be on chip or nearby (external)
- Faster for CPU to access than main memory

## 6. Video Random Access memory

- Holds data to be displayed on computer screen
- Has two data paths allowing READ and WRITE to occur at the same time
- A system's amount of VRAM relates to the number of colours and resolution
- A graphics card may have its own VRAM chip on board

## Read only memory (ROM)

- ✓ ROM holds programs and data **permanently** even when computer is switched off
- ✓ Data can be read by the CPU in any order so ROM is also **direct access**
- ✓ The contents of ROM are fixed at the time of manufacture.
- ✓ Stores a program called the **bootstrap loader** that helps start up the computer
- ✓ Access time of between 10 and 50 nanoseconds.

These ROMs are called read only because they cannot be modified by the casual user (and some types cannot be modified at all). ROMs have traditionally been used in computer systems to store configuration data, such as bootstrap or BIOS code, which requires fast access.

## Types of ROM

### 1. Programmable Read Only Memory (PROM)

- Empty of data when manufactured
- May be permanently programmed by the user

### 2. Erasable Programmable Read Only Memory (EPROM)

- Can be programmed, erased and reprogrammed
- The EPROM chip has a small window on top allowing it to be erased by shining ultra-violet light on it
- After reprogramming the window is covered to prevent new contents being erased
- Access time is around 45 – 90 nanoseconds

### 3. Electrically Erasable Programmable Read Only Memory (EEPROM)

- Reprogrammed electrically **without** using ultraviolet light
- Must be removed from the computer and placed in a special machine to do this
- Access times between 45 and 200 nanoseconds

#### 4. Flash ROM

- Similar to EEPROM
- However, can be reprogrammed while still in the computer
- Easier to upgrade programs stored in Flash ROM
- Used to store programs in devices e.g. modems
- Access time is around 45 – 90 nanoseconds

### INPUT/OUTPUT (Syllabus)

#### Input Devices

The devices which are used to give data and instructions to the computer are called Input Devices. Various types of input devices can be used with the computer depending upon the type of data you want to enter in the computer, e.g., keyboard, mouse, joystick, light pen, etc.

#### Output Devices

The devices which are used to display the results or information are called Output Devices. You can view the output on the monitor or you can print it on a paper using a printer. Monitor and the printer are the commonly used output devices.

### HARDWARE AND SOFTWARE (Syllabus)

Hardware refers to the physical elements of a computer. This is also sometime called the machinery or the equipment of the computer. Examples of hardware in a computer are the keyboard, the monitor, the mouse and the central processing unit. However, most of a computer's hardware cannot be seen; in other words, it is not an external element of the computer, but rather an internal one, surrounded by the computer's casing (tower). A computer's hardware is comprised of many different parts, but perhaps the most important of these is the motherboard. The motherboard is made up of even more parts that power and control the computer.

In contrast to software, hardware is a physical entity. Hardware and software are interconnected, without software; the hardware of a computer would have no function. However, without the creation of hardware to perform tasks directed by software via the central processing unit, software would be useless.

#### Software

[Software](#), commonly known as programs or apps, consists of all the instructions that tell the hardware how to perform a task. These instructions come from a software developer in the form that will be accepted by the *platform* (operating system + CPU) that they are based on. For example, a program that is designed for the Windows operating system will only work for that specific operating system.

Software is capable of performing many tasks, as opposed to hardware which can only perform mechanical tasks that they are designed for. Software provides the means for accomplishing many different tasks with the same basic hardware. Practical computer systems divide software systems into two major classes:

- **System software:** Helps run the computer hardware and computer system itself. System software includes operating systems, device drivers, diagnostic tools and more. System software is almost always pre-installed on your computer.
- **Application software:** Allows users to accomplish one or more tasks. It includes word processing, web browsing and almost any other task for which you might install software. (Some application software is pre-installed on most computer systems.)

Software is generally created (written) in a high-level programming language, one that is (more or less) readable by people. These high-level instructions are converted into "machine language" instructions, represented in binary code, before the hardware can "run the code". When you install software, it is generally already in this machine language, binary, form.

Ref-<http://cs.sru.edu>

### UNDERSTAND BIT, BYTE, KB, MB, GB AND THEIR RELATIONS TO EACH OTHER (Syllabus)

Computers store information in the form of "1" and "0"s in different types of storages such as memory, hard disk, USB drives etc. The most common digital data storage unit is byte which is 8 bits.

Computer data is expressed as bytes, kilobytes, megabytes as it is in the metric system, but 1 kilobyte is 1024 bytes not 1000 bytes.

Data storage units are: bit, byte, kilobyte (kb), megabyte (mb), gigabyte (gb), terabyte (tb), Petabyte and Exabyte.

The following table explains the main memory storage units –

| S.No. | Unit & Description                                                                                                                                                                                                                                                                                                                                                           |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <b>Bit (Binary Digit)</b><br>A binary digit is logical 0 and 1 representing a passive or an active state of a component in an electric circuit.                                                                                                                                                                                                                              |
| 2     | <b>Nibble</b><br>A group of 4 bits is called nibble.                                                                                                                                                                                                                                                                                                                         |
| 3     | <b>Byte</b><br>A group of 8 bits is called byte. A byte is the smallest unit, which can represent a data item or a character.                                                                                                                                                                                                                                                |
| 4     | <b>Word</b><br>A computer word, like a byte, is a group of fixed number of bits processed as a unit, which varies from computer to computer but is fixed for each computer.<br>The length of a computer word is called word-size or word length. It may be as small as 8 bits or may be as long as 96 bits. A computer stores the information in the form of computer words. |

|                                                                                                                                                                                                                          |                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 4 bits = 1 nibble<br>8 bits = 1 Byte<br>1024 Bytes = 1 KiloByte<br>1024 KiloBytes = 1 MegaByte<br>1024 MegaBytes = 1 GigaByte<br>1024 GigaBytes = 1 TeraByte<br>1024 TeraBytes = 1 PetaByte<br>1024 PetaByte = 1 ExaByte | Computers count by base 2:<br>$2^1 = 2$<br>$2^2 = 2*2 = 4$<br>$2^3 = 2*2*2 = 8$<br>$2^{10} = 1,024$<br>$2^{20} = 1,048,576$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|

Ref- <https://www.tutorialspoint.com>

## OPERATING SYSTEM (Syllabus)

### Definition 1

- An operating system is a program that manages the computer hardware.
- It also provides a basis for application programs and acts as an intermediary between the computer user and computer hardware.

### Definition 2

- ☐ An OS is a program which acts as an *interface* between computer system users and the computer hardware.
- ☐ It provides a user-friendly environment in which a user may easily develop and execute programs.
- ☐ Otherwise, hardware knowledge would be mandatory for computer programming.
- ☐ So, it can be said that an OS hides the complexity of hardware from uninterested users.

The OS manages hardware resources and allocates them to specific programs and users.

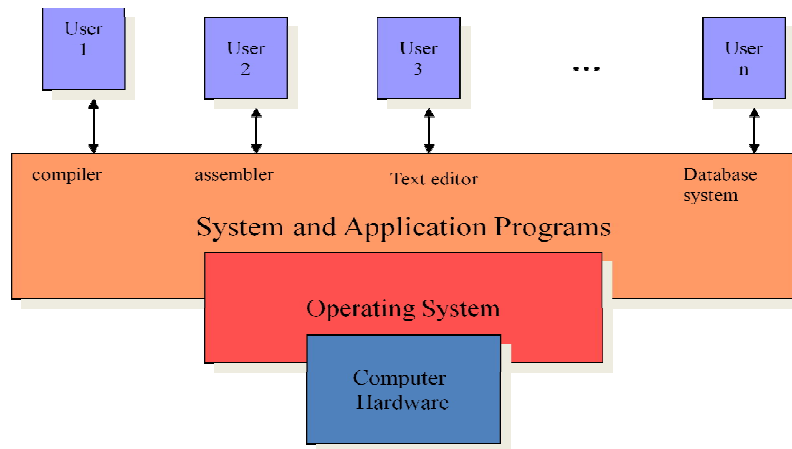
With the management of the OS, a programmer is rid of difficult hardware considerations.

Another aspect for the usage of OS is that; it is used as a *predefined library* for hardware-software interaction. This is why, system programs apply to the installed OS since they cannot reach hardware directly.

- **The computer system can be divided roughly into four components:**

1. The hardware
2. Software –Application/System
3. Data

#### 4. The Users



#### Task/ Function/ Characteristics of Operating System

Here is a list of some of the most prominent characteristic features of Operating Systems –

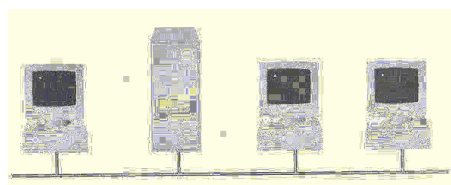
- **Memory Management** – Keeps track of the primary memory, i.e. what part of it is in use by whom, what part is not in use, etc. and allocates the memory when a process or program requests it.
- **Processor Management** – Allocates the processor (CPU) to a process and deallocates the processor when it is no longer required.
- **Device Management** – Keeps track of all the devices. This is also called I/O controller that decides which process gets the device, when, and for how much time.
- **File Management** – Allocates and de-allocates the resources and decides who gets the resources.
- **Security** – Prevents unauthorized access to programs and data by means of passwords and other similar techniques.
- **Job Accounting** – Keeps track of time and resources used by various jobs and/or users.
- **Control over System Performance** – Records delays between the request for a service and from the system.
- **Interaction with the Operators** – Interaction may take place via the console of the computer in the form of instructions. The Operating System acknowledges the same, does the corresponding action, and informs the operation by a display screen.
- **Error-detecting Aids** – Production of dumps, traces, error messages, and other debugging and error-detecting methods.
- **Coordination between Other Software and Users** – Coordination and assignment of compilers, interpreters, assemblers, and other software to the various users of the computer systems.

#### COMPUTER NETWORK (Syllabus)

##### What is a Network?

A network consists of 2 or more computers connected together, and they can communicate and share resources (e.g. information)

Connections between computing devices can be physical using wires or cables or wireless using radio waves or infrared signals.



##### Why Networking?

- Sharing information — i.e. data communication
- Sharing hardware or software
- Centralize administration and support

##### How many kinds of Networks?

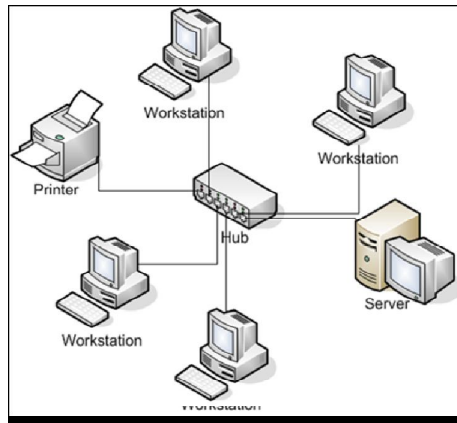
Depending on one's perspective, we can classify networks in different ways

- Based on transmission media: Wired (UTP, coaxial cables, fiber-optic cables) and Wireless
- Based on network size: LAN, MAN and WAN
- Based on management method: Peer-to-peer and Client/Server(How Networks Are Structured)
- Based on topology (connectivity): Bus, Star, Ring

### Local Area Network (LAN)

- Small network, short distance
- Spans a relatively small area.
- A room, a floor, a building, a computer lab.
- Limited by no. of computers and distance covered
- Usually one kind of technology throughout the LAN
- Serve a department within an organization

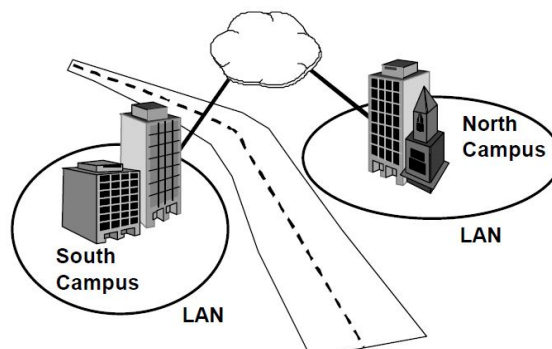
LANs are the most common form of networks found in most small businesses and schools. Most LANs connect workstations and personal computers. Each node (individual computer) in a LAN has its own CPU with which it executes programs, but it is also able to access data and devices anywhere on the LAN. This means that many users can share expensive devices, such as laser printers, as well as data. Users can also use the LAN to communicate with each other, by sending e-mail or engaging in chat sessions.



### Metropolitan Area Network - MAN

Short for Metropolitan Area Network, a data network designed for a town or city. In terms of geographic breadth, MANs are larger than local-area networks (LANs), but smaller than wide-area networks (WANs). MANs are usually characterized by very high-speed connections using fiber optical cable or other digital media. Using high-speed transmission, MANs connect smaller networks within a campus, a city, and/or neighboring suburbs.

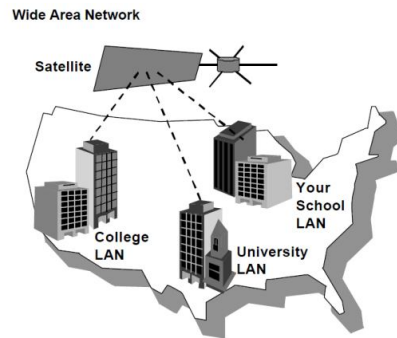
Metropolitan Area Network



### Wide Area Network (WAN)

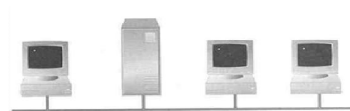
A network that uses long-range telecommunication links to connect 2 or more LANs/computers housed in different places far apart. Computers connected to a wide-area network are often connected through public

networks, such as the telephone system. They can also be connected through leased lines or satellites. The largest WAN in existence is the Internet. For example, for a large company the head quarters might be at Delhi and regional branches at Bombay, Madras, Bangalore and Calcutta. Here regional centers are connected to head quarters through WAN. The distance between computers connected to WAN is larger. Therefore the transmission medium used is normally telephone lines, microwaves and satellite links.

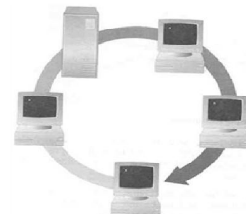


### Network Topology — 3 basic types

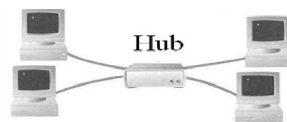
#### Bus Topology



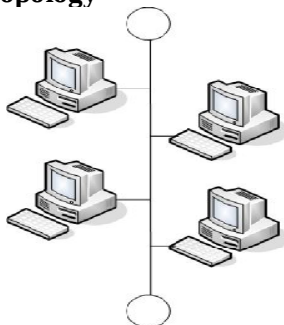
#### Ring Topology



#### Star Topology



#### Bus Topology



- Simple and low-cost
- A single cable called a trunk (backbone, segment)
- Only one computer can send messages at a time
- Passive topology - computer only listen for, not regenerate data
- In this configuration every computer (node) shares the networks total bus capacities.
- In this configuration adding more computers will reduce the access speed on the network.
- Each computer communicates to other computers on the network independently this is referred to as PEER-TO-PEER networking

#### How a Bus Peer to Peer Network Works

- All computers on a network have a distinct address just like your house does
- a message would be send from one computer with the address of another computer attached to the message

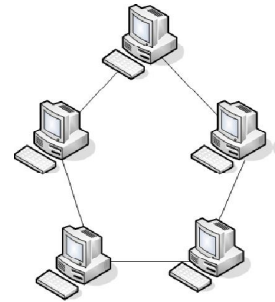
- The message is broadcasted to all the computers on the network until the addressed PC accepts the message

### Problems in Bus Topology

- One of the main problems with this type of network is that it is not very fault tolerant, a break or defect in the bus would affect the whole network

### Ring Topology

- In Ring topology each node is connected to the two nearest nodes so the entire network forms a circle
- Data only travels in one direction on a Ring network
- Every computer serves as a repeater to boost signals
- Typical way to send data: Token passing  
Only the computer who gets the token can send data



### How Ring Topology works

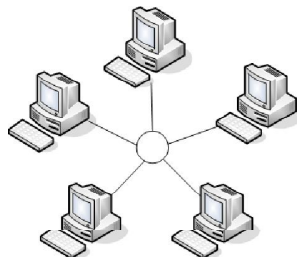
- a node has information to send to another computer on the network so it sends the information out on the network to the PC it is connected to, if the information is for this PC (the recipients NIC address is attached to the message, which is like putting an address on an envelope) then the PC accepts the data
- otherwise it passes the information on to the next PC by repeating the data back out on the line
- This method of repeating the data helps keep the integrity of the data readable by other computers

### Disadvantages

- Difficult to add computers
- More expensive
- If one computer fails, whole network fails

### Star Topology

- In a Star topology every node is connected through a central device such as a Hub, Switch or Router
- Compared to a Ring or Bus topology a Star topology requires that more thought be put into its setup
- Each computer has a cable connected to a single point
- More cabling, hence higher cost
- All signals transmission through the hub; if down, entire network down
- Depending on the intelligence of hub, two or more computers may send message at the same time



### The Good and Bad of a Star Network

- The upside of a star network is that if any one cable fails then only the node connected on that cable would be affected
- Another positive point to this type of network is that it is very simple to join two star networks together by connecting their central devices to each other

## UNIT 1 PART B

### Algorithm & Flowcharts

A computer is a useful tool for solving a great variety of problems. To make a computer do anything (i.e. solve a problem), you have to write a computer program.

A typical programming task can be divided into two phases:

#### ■ Problem solving phase

- ☐ Produce an ordered sequence of steps that describe solution of problem.
- ☐ this sequence of steps is called an **algorithm**

#### ■ Implementation phase

- ☐ Implement the program in some programming language.

**Program Design** consists of the steps a programmer should do before they start coding the program in a specific language. Proper program design helps other programmers to maintain the program in the future.

### Algorithm

The sequence of steps to be performed in order to solve a problem by the computer is known as an algorithm. In mathematics, computer science, and related subjects, an algorithm is a finite sequence of steps expressed for solving a problem.

An algorithm can be defined as “a process that performs some sequence of operations in order to solve a given problem”. Algorithms are used for calculation, data processing, and many other fields.

### Advantages of Using Algorithm

- Identification of the processes, major decision points, and variables necessary to solve the problem.
- A benefit of the use of an algorithm comes from the **improvement** it makes possible.
- It is independent of programming language.
- It easy to first develop an algorithm, & then convert it into a flowchart & then into a computer program.

### Flowchart

Flowchart is a graphical or symbolic representation of an algorithm. It is the diagrammatic representation of the step-by-step solution to a given problem.

A Flowchart is a type of diagram (graphical or symbolic) that represents an algorithm or process. Each step in the process is represented by a different symbol and contains a short description of the process step. The flow chart symbols are linked together with arrows showing the process flow direction.

A Flowchart describes what operations (and in what sequence) are required to solve a given problem.

### Advantages of Using Flowcharts

The benefits of flowcharts are as follows:

- **Communication:** Flowcharts are better way of communicating the logic of a system to all concerned.
- **Effective analysis:** With the help of flowchart, problem can be analyzed in more effective way.
- **Proper documentation:** Program flowcharts serve as a good program documentation, which is needed for various purposes.
- **Efficient Coding:** The flowcharts act as a guide or blueprint during the systems analysis and program development phase.
- **Proper Debugging:** The flowchart helps in debugging process.
- **Efficient Program Maintenance:** The maintenance of operating program becomes easy with the help of flowchart. It helps the programmer to put efforts more efficiently on that part.

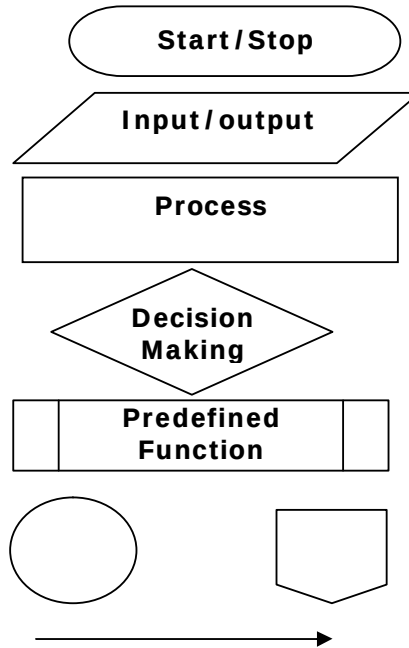
## PROGRAM CONTROL STRUCTURES

Program statements that affect the order in which statements are executed, or that affect whether statements are executed, are called control structures. There are three control structures.

- **Sequence** construct is a linear progression where one task is performed sequentially after another.
- **Selection (Decision)** structure allows the program to make a choice between two alternate paths, whether it is true or false.
- **Repetition (Looping)** causes an interruption in the normal sequence of processing and directs the system to loop back to a previous statement.

## Flowchart Symbols

1. Start/Stop/Terminator
2. Read Input/Write Output
3. Process (Initialization, Calculation)
4. Decision Making
5. Predefined Function
6. Connectors
7. Control flow



## Decision Structures

### ***IS Structure***

Is (True) Then  
     Begin  
         Zero or more Statements  
     End

### ***IS ELSE Structure***

Is (True) Then  
     Begin  
         Zero or more Statements  
     End  
 Else  
     Begin  
         Zero or more Statements  
     End

### ***IS ELSE-IF ELSE Structure***

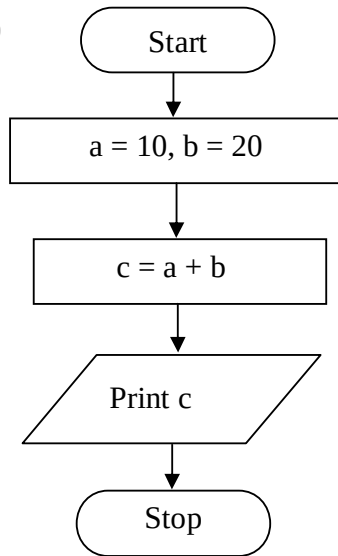
Is (True) Then  
     Begin  
         Zero or more Statements  
     End  
 Else if (True) Then  
     Begin  
         Zero or more Statements  
     End  
 Else  
     Begin  
         Zero or more Statements  
     End

Q1- Algorithm to perform addition of two numbers

Step 1 -  $a = 10, b = 20$

Step 2 -  $c = a + b$

Step 3 - print  $c$

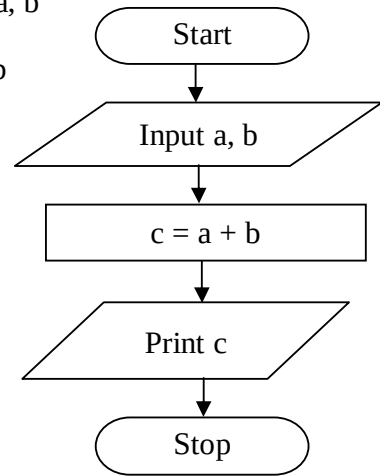


Q2 – Algorithm to perform addition of two numbers

Step 1 - Input  $a, b$

Step 2 -  $c = a + b$

Step 3 - print  $c$

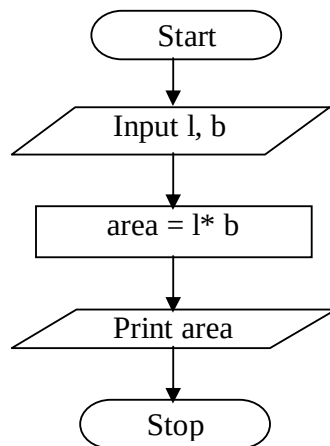


Q3- Algorithm to find area of rectangle.

Step1 - Input  $l, b$

Step2 -  $\text{area} = l * b$

Step3 - Print area

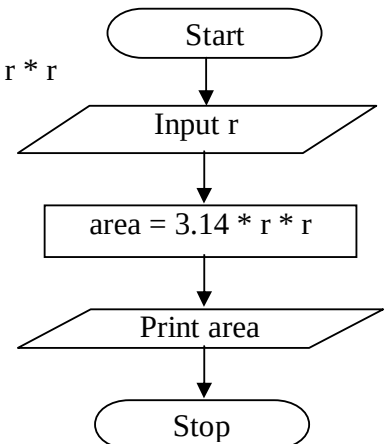


Q4- Algorithm to find area of circle.

Step1 - Input  $r$

Step2 -  $\text{area} = 3.14 * r * r$

Step3 - Print area

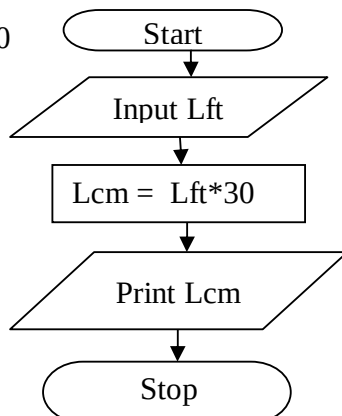


Q5- Algorithm to convert length in feet to cm.

Step 1- Input  $Lft$

Step 2-  $Lcm = Lft * 30$

Step 3- Print  $Lcm$

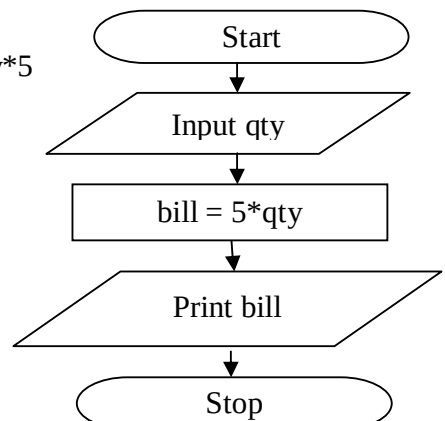


Q6- Algorithm to generate bill for shopkeeper who sales mango at 5 rupees per mango

Step1 – Input  $qty$

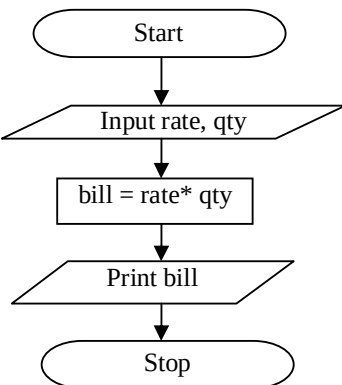
Step2 –  $\text{bill} = qty * 5$

Step3 – Print bill



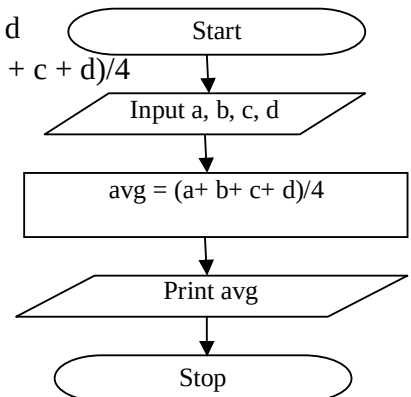
Q7- Algorithm to generate bill for a shopkeeper who sales mango

Step 1- Input rate, qty  
Step 2- bill = rate \* qty  
Step 3- Print bill



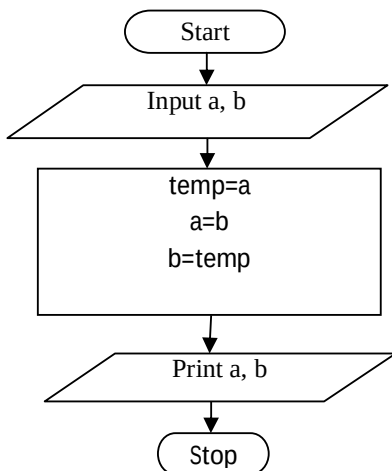
Q8- Algorithm to find average of 4 integers

Step1- Input a, b, c, d  
Step2 - avg = (a + b + c + d)/4  
Step3- Print avg



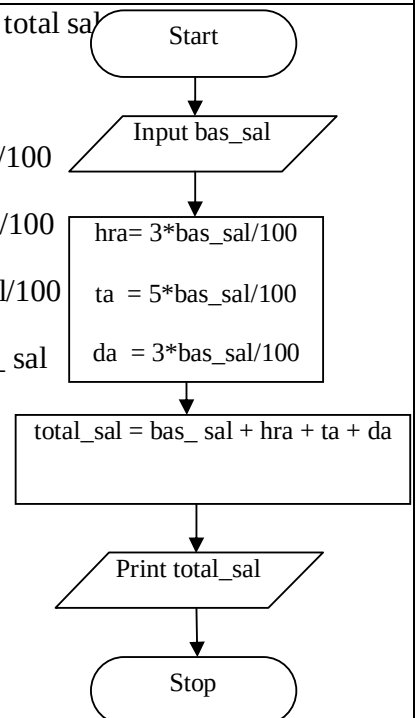
Q9-Algorithm to swap two numbers using third variable.

Step 1- Input a, b  
Step 2- temp=a  
a = b  
b = temp  
Step 3- print a, b



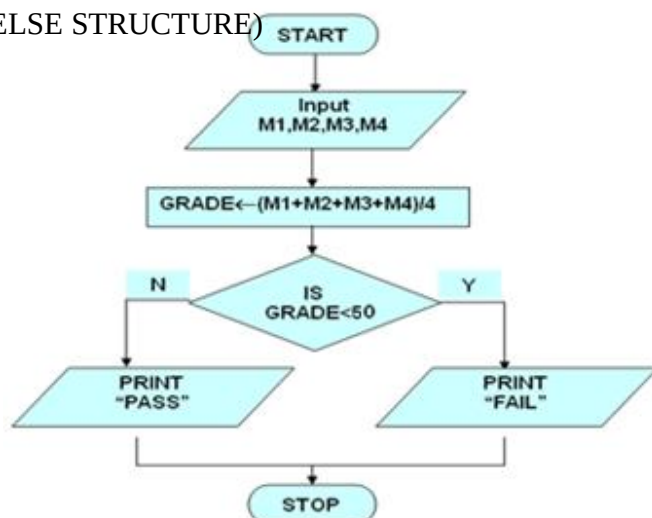
Q10-Algorithm to find total sal

Step 1- Input bas\_sal  
Step 2- hra= 3\*bas\_sal/100  
ta = 5\*bas\_sal/100  
da = 3\*bas\_sal/100  
Step 3- total\_sal = bas\_sal  
+ hra + ta + da  
Step 4- print total\_sal



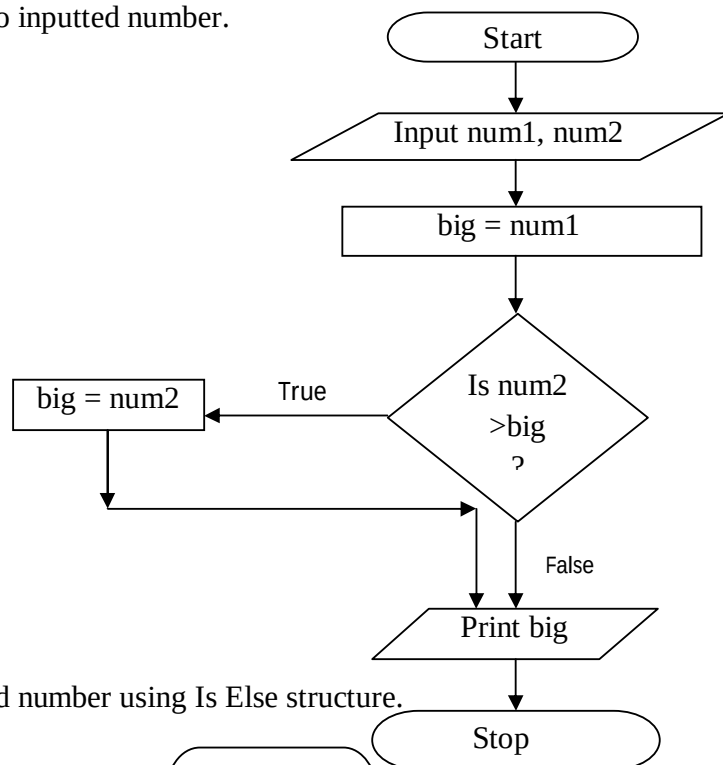
Q11 CALCULATE RESULT BASED ON GRADE (i.e. AVERAGE OF 4 SUBJECT'S MARKS).PRINT PASS IF GRADE >=50 OTHERWISE PRINT FAIL (IS ELSE STRUCTURE)

**STEP 1.** Input M1,M2,M3,M4  
**STEP 2.** GRADE = (M1+M2+M3+M4)/4  
**STEP 3.** Is (GRADE <50) Then  
Begin  
Print "FAIL"  
End  
Else  
Begin  
Print "PASS"  
End



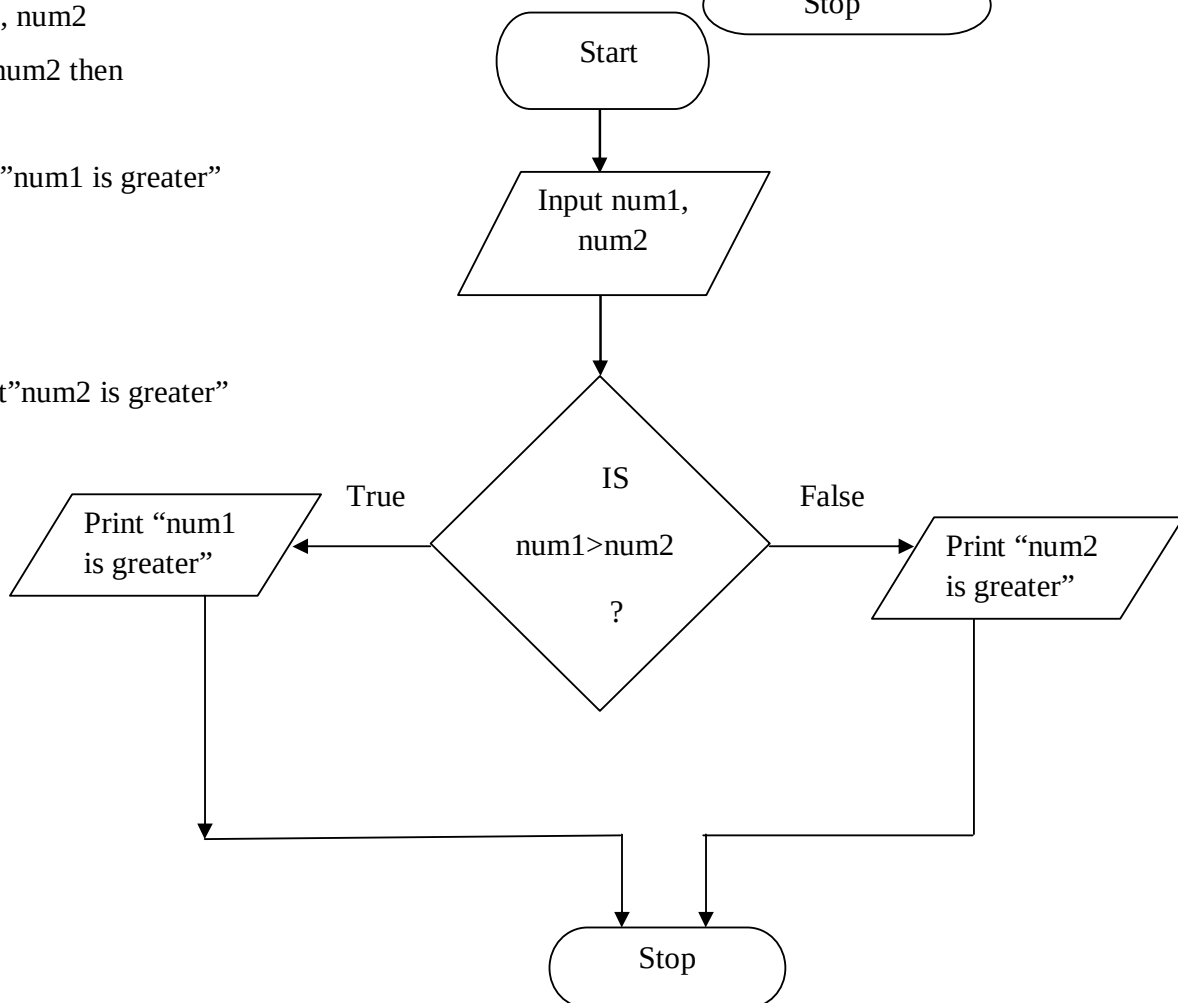
Q12- Write an algorithm to find greater of two inputted number.

Step 1- Input num1, num2  
 Step 2- big = num1  
 Step 3- Is num2 > big then  
     Begin  
     big = num2  
     End  
 Step 4- Print big



Q13. Algorithm to find greater of two inputted number using Is Else structure.

Step 1- Input num1, num2  
 Step 2- Is num1 > num2 then  
     Begin  
     Print "num1 is greater"  
     End  
 Else  
     Begin  
     Print "num2 is greater"  
     End



Q14- Algorithm to find given number is +ve, -ve or zero

Step 1- Input num

Step 2- Is num > 0 then

Begin

Print "number is positive"

End

Else if num < 0 then

Begin

Print "number is negative"

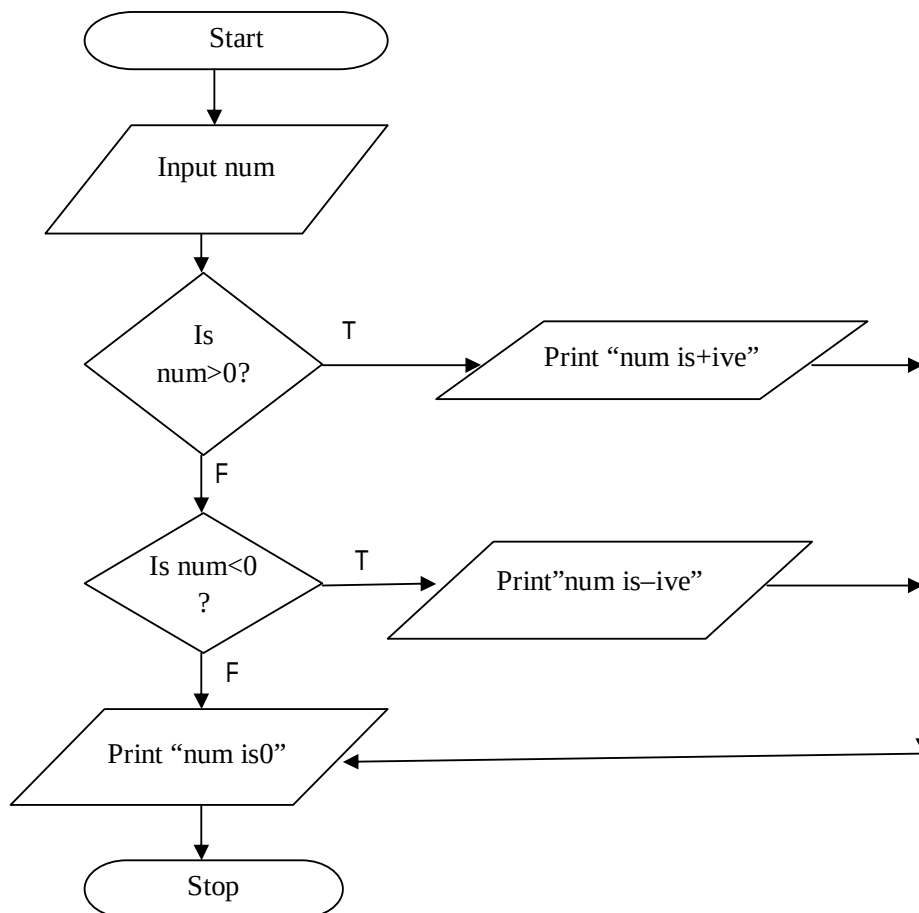
End

Else

Begin

Print "number is zero"

End



Q 15- Algorithm to find biggest of 4 numbers

Step 1-Input num1, num2, num3, num4

Step2- big=num1

Step3-Is (num2>big) then

Begin

big=num2

End

Is (num3>big) then

Begin

big=num3

End

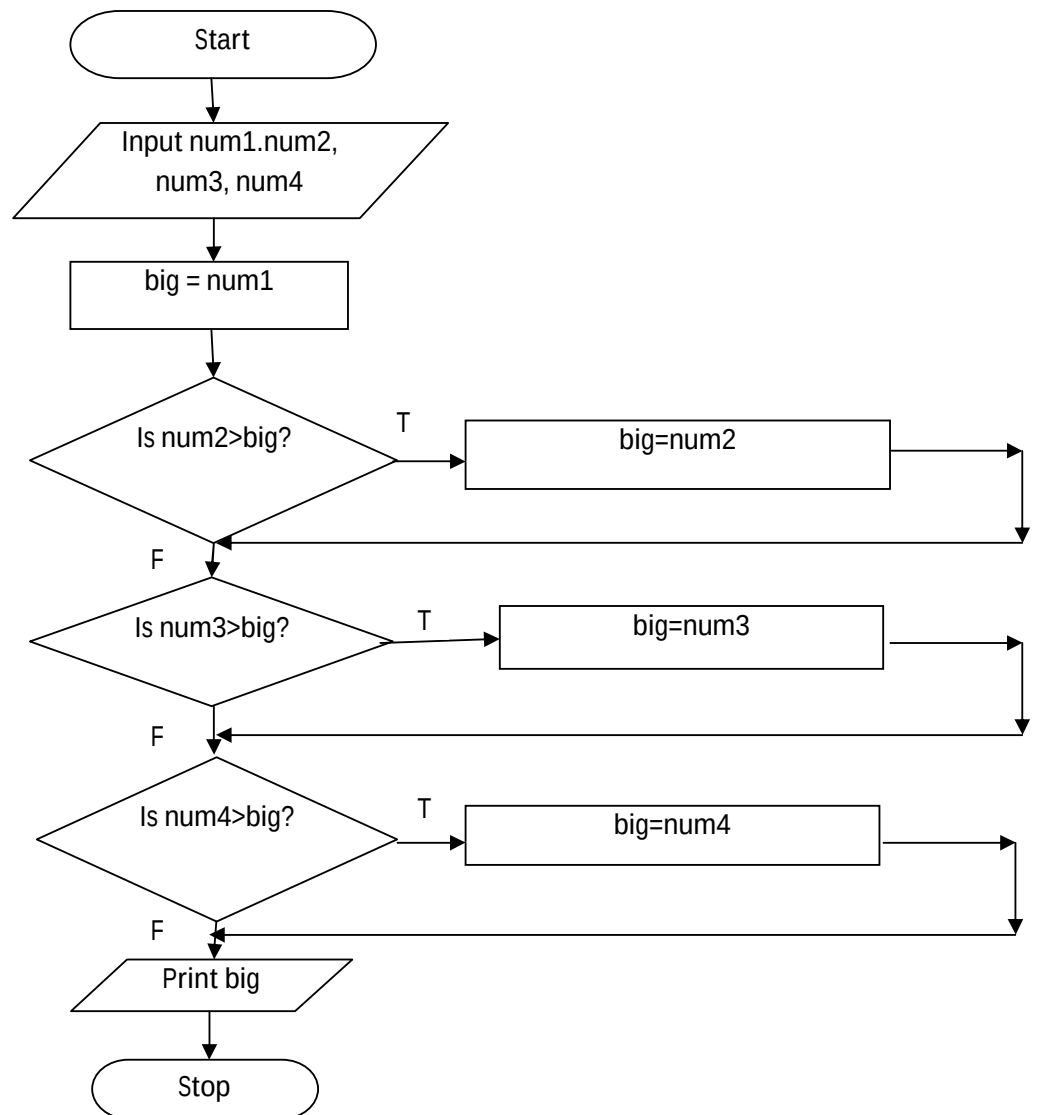
Is (num4>big) then

Begin

big=num4

End

Step4-Print big



Q16-A student takes an entrance test consisting of four subjects. The percentage is to be calculated. The maximum marks per subject are hundred. If the percentage is more than 80 the student is graded as excellent. If the percentage is more than 70 the student is graded as distinction and for more than 60 students is graded as eligible. For any other percentage he is graded as ineligible.(is else-if else structure)

**STEP 1.** Read S1, S2, S3, S4

**STEP 2.** Percent =  $(S1+S2+S3+S4)*100/400$

**STEP 3.** Is (Percent > 80) Then

Begin

Print "Excellent"

End

Else if (Percent > 70) Then

Begin

Print "Distinction"

End

Else if (Percent > 60) Then

Begin

Print "Eligible"

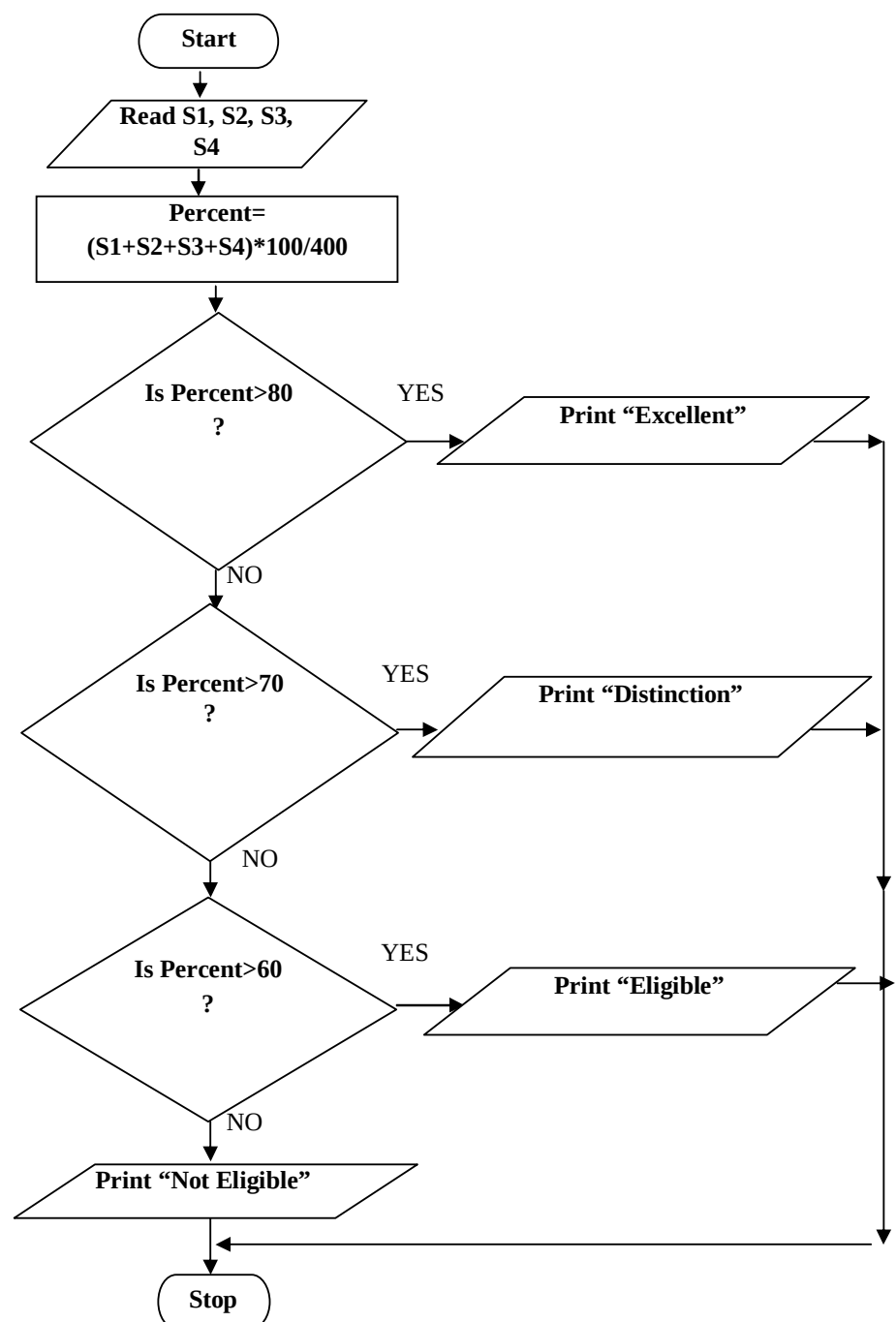
End

Else

Begin

Print "Not Eligible"

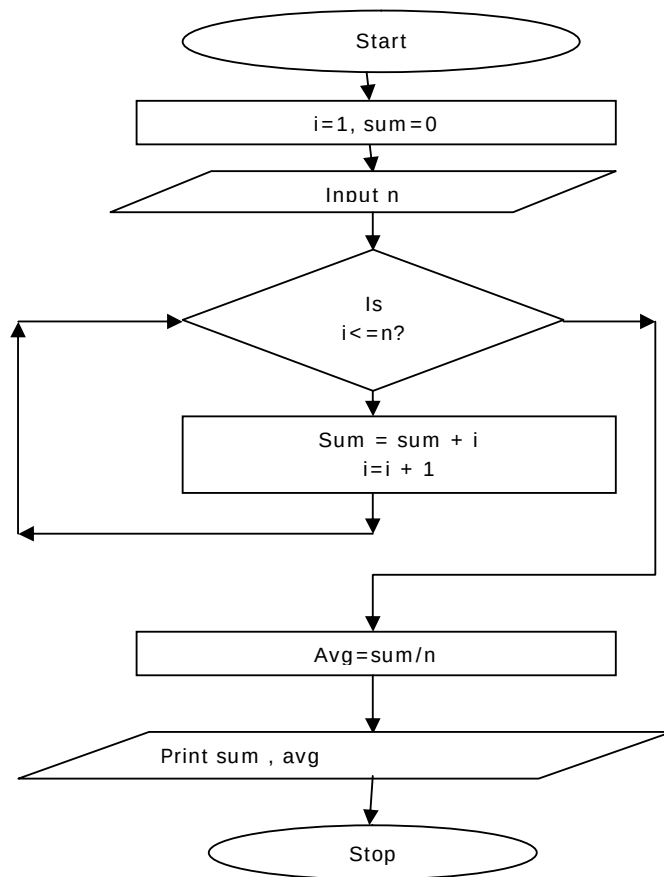
End



Q17 Write a C program to find sum and average of first n natural numbers using while loop

**Algorithm**

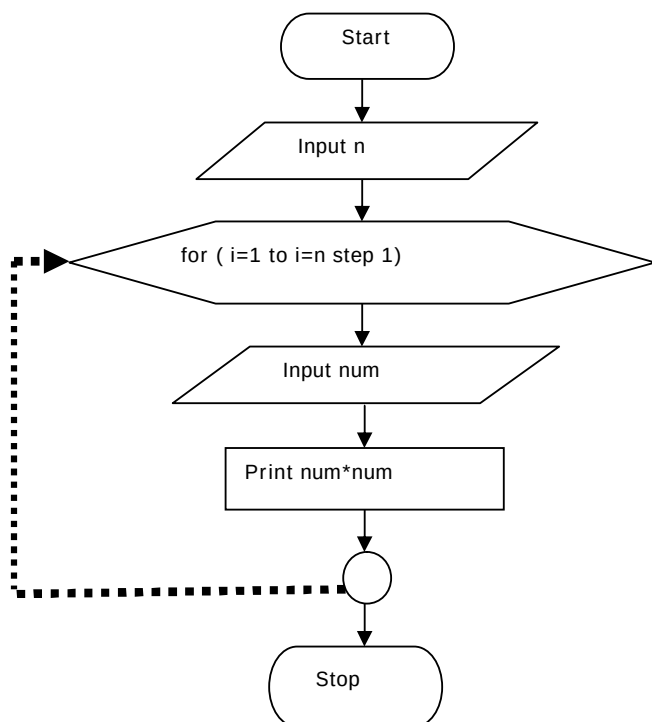
- Step 1.  $i=1, \text{sum}=0$   
 Step 2. Input n  
 Step 3. while ( $i \leq n$ ) repeat step 4,5  
 Step 4.  $\text{Sum} = \text{sum} + i$   
 Step 5.  $i=i+1$   
 Step 6.  $\text{avg} = \text{sum} / n$   
 Step 7. Print sum , avg



Q18 Write a C program to Print Square of n numbers, where value of n is inputted by the user. (For loop)

**Algorithm**

- Step 1: Input n  
 Step 2: Repeat for  $i=1,2,3,\dots,n$   
     Begin  
         Input num  
         Print num\*num  
     End



Q19 Find the sum of even and odd numbers between x and y.

(While loop)

### Algorithm

To find the sum of even and odd numbers between x and y

Step 1. Input x,y

Step 2. i= x

Step 3. While i<=y repeat step 4, 5

Step 4 If (i%2==0) then

Begin

sume = sume+i

End

Else

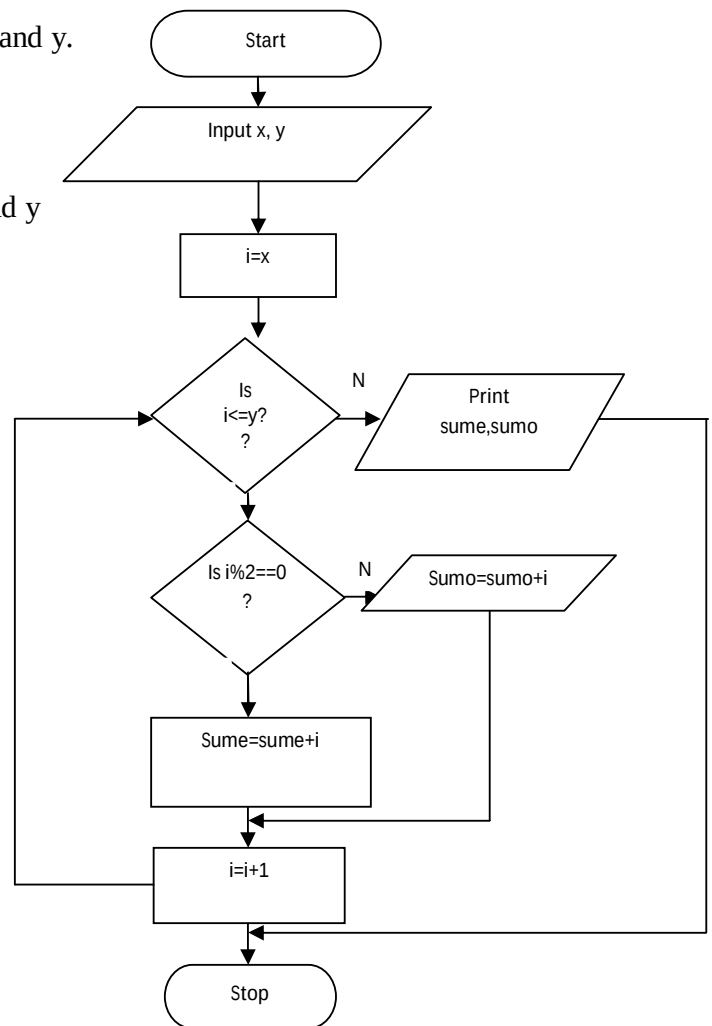
Begin

sumo = sumo+i

End

Step 5. i= i+1

Step 6. Print sume , sumo



Heron's formula to calculate area of triangle

$$A = \sqrt{s(s-a)(s-b)(s-c)},$$

Where  $s = (a + b + c)/2$  and a, b and c are sides of triangle

|  |                                                                                                             |
|--|-------------------------------------------------------------------------------------------------------------|
|  | <b>Triangle</b><br>Area = $\frac{1}{2} \times b \times h$<br>b = base<br>h = vertical height                |
|  | <b>Rectangle</b><br>Area = $w \times h$<br>w = width<br>h = height                                          |
|  | <b>Trapezoid (US)</b><br><b>Trapezium (UK)</b><br>Area = $\frac{1}{2}(a+b) \times h$<br>h = vertical height |
|  | <b>Ellipse</b><br>Area = $\pi ab$                                                                           |

Note: h is at right angles to b:

|  |                                                                                                             |
|--|-------------------------------------------------------------------------------------------------------------|
|  | <b>Square</b><br>Area = $a^2$<br>a = length of side                                                         |
|  | <b>Parallelogram</b><br>Area = $b \times h$<br>b = base<br>h = vertical height                              |
|  | <b>Circle</b><br>Area = $\pi \times r^2$<br>Circumference = $2 \times \pi \times r$<br>r = radius           |
|  | <b>Sector</b><br>Area = $\frac{1}{2} \times r^2 \times \theta$<br>r = radius<br>$\theta$ = angle in radians |

Area of sphere=  $4 \pi r^2$

Area of cylinder=  $2 \pi r^2 + 2 \pi r h$

Volume of cylinder=  $\pi r^2 h$

Circumference of circle=  $2 \pi r$

The circumference of a sphere is:  $C = 2\pi R$

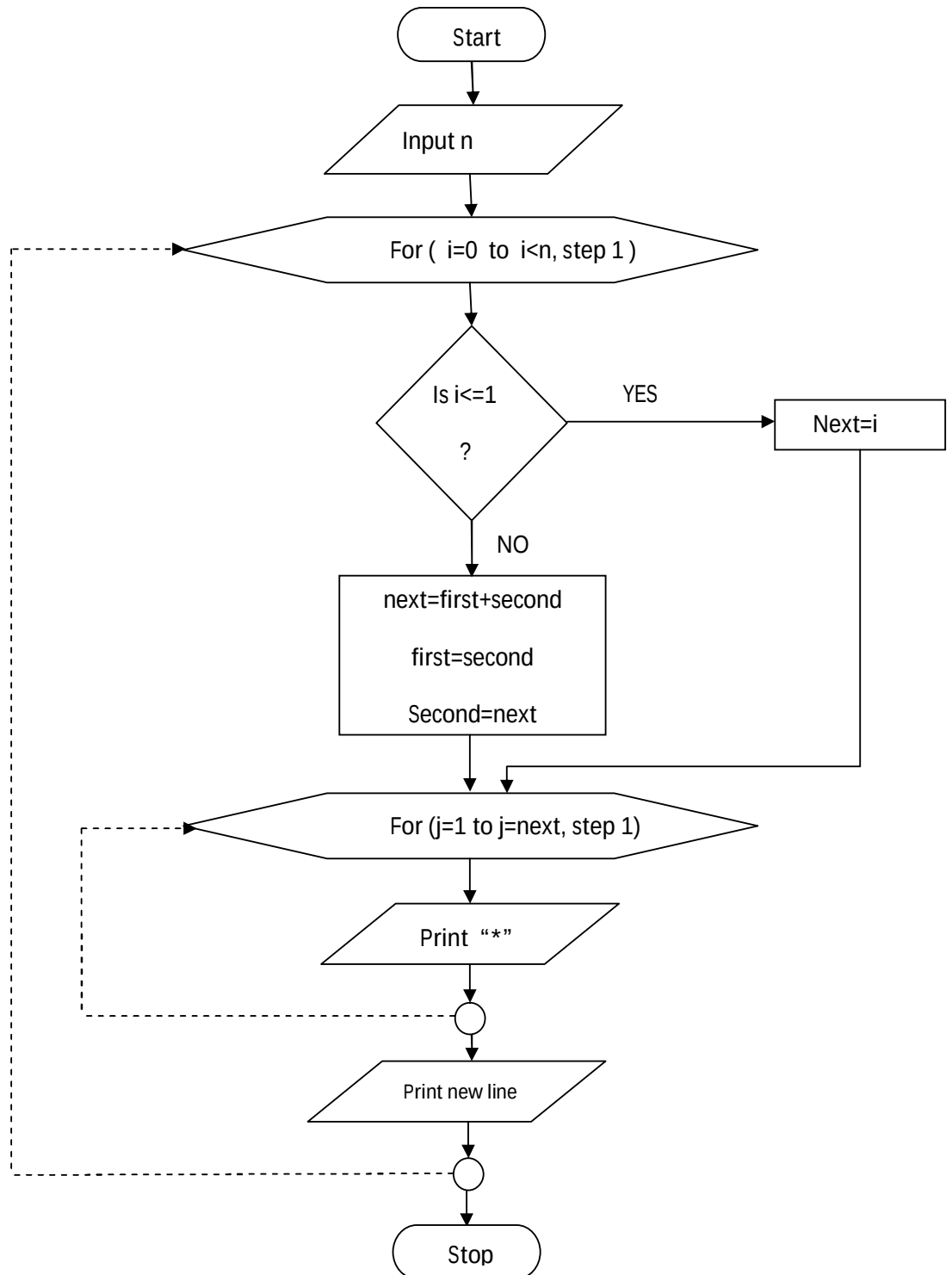
The volume of a sphere is:

$$V = \frac{4\pi R^3}{3}$$

## Nested For loop

### . Draw Fibonacci Pattern

\*  
\*  
\*\*  
\*\*\*  
\*\*\*\*  
\*\*\*\*\*  
\*\*\*\*\*  
\*\*\*\*\*  
Up to n rows



## UNIT 2

### Brief History of 'C'

- ALGOL
- BCPL Martin Richards
- B Derivative BCPL (Ken Thompson)
- Early versions of UNIX OS 1970
- 1972 C Dennis Ritchie Bell Labs
- 1978 C Programming Language Ritchie and Kernighan
- 1983 ANSI committee formed
- 1988 ANSI Standard was Defined

### C character set

0-9 digits  
a-z alphabets  
A-Z alphabets  
Special characters  
white spaces (new line , space, tab)

### C tokens:

- C tokens are the basic buildings blocks in C language which are constructed together to write a C program.
- Each and every smallest individual unit in a C program is known as C token

C tokens are of six types. They are,

1. Keywords (eg: int, while),
2. Identifiers (eg: main, total),
3. Constants (eg: 10, 20),
4. Strings (eg: "total", "hello"),
5. Special symbols (eg: (), {}),
6. Operators (eg: +, /, -, \*)

### C Keywords:

Keywords are the words whose meaning has already been explained to the C compiler. The keywords cannot be used as variable names because if we do so, we are trying to assign a new meaning to the keyword, which is not allowed by the computer. There are only 32 keywords available in C. The following table shows the list of keywords:

|          |        |          |          |
|----------|--------|----------|----------|
| auto     | double | Int      | struct   |
| break    | else   | Long     | switch   |
| case     | enum   | register | typeof   |
| char     | extern | Return   | union    |
| const    | float  | Short    | unsigned |
| continue | for    | signed   | void     |
| default  | goto   | Sizeof   | volatile |
| do       | if     | Static   | while    |

### Identifiers:

Any used defined name given to the program element is called as identifier.  
(i.e Program elements are identified by program with the identifier name)

#### Some facts about identifier:

It is name given to program element.

2. Identifiers are the names given by the programmer/user.
3. An identifier is used for any variable, function, data definition etc.
4. We can give any valid name to the identifier. Consider following rules:

#### Rules for identifiers

1. Only Alphabets, Digits and Underscores are permitted.
2. Identifier name cannot start with a digit.
3. Key words cannot be used as a name.
4. Upper case and lower case letters are distinct.
5. Special Characters are not allowed
6. we can not use space in identifiers

### **Constants:**

A constant is a quantity that does not change. This quantity can be stored at a location in the memory of the computer. Naturally the contents of the variable can change. For example in the equation:

$$3X + Y = 20$$

since 3 and 20 cannot change, they are called constants, whereas the quantities X and Y can vary or change and are hence called variables.

**Integer constants:** An integer constant must have at least one digit. It must not have a decimal point. It could be either positive or negative. If no sign precedes an integer constant it is assumed to be positive. No commas or blanks are allowed within an integer constant. The allowable range for an integer constant is -32768 to +32767.

**For example:**

243 , +476 , -8000 , -7605

**Real Constants:** Real constants are often called Floating Point constants. The real constants could be written in two forms - fractional form or exponential form. A real constant must have at least one digit. It must have a decimal point. It could be either positive or negative. Default sign is positive. The part appearing before 'e' is called mantissa and the part following 'e' is called the exponent. In place of a small case 'e' a capital 'E' can also be used.

- Range of real constants expressed in exponential form is -3.4e38 to 3.4e38.

**For example:**

+3.2e-5 , 4.1e8 , -0.2e+3 , -3.2e-5

**Character constants:** A character constant is a single alphabet, a single digit or a single special symbol enclosed within a pair of single inverted commas. Both the inverted commas should point to the left.

The maximum length of a character constant can be 1 character.

**Examples:** 'A' 'T' '5'

The valid range of character constant is -128 to +127.

'A' and 65 are one and the same, since when we say 'A' it is replaced by its ASCII value, which is 65.

**Logical Constants:** A logical constant can take either of two values: truth or falsity. In C, a non-zero value is always treated as truth whereas a zero is treated as falsity. Hence, values like 32, -45, 3.12 etc. are treated as logical constants having a 'true' value, whereas a 0 is treated as a logical constant with a 'false' value. The logical constants are very useful in evaluating logical expressions and complex conditions.

**String Constants:** A collection of characters enclosed within a pair of double inverted commas is treated as a string constant.

**Examples:**

"Hello"

"44-A, Gokulpeth, Nagpur"

"34.567"

In the above example, though 34.567 appear to be a real constant, once it is enclosed within inverted commas, it is treated as a string constant.

### **Variables:**

In C, a quantity that may vary during program execution is called a variable. Variable names are names given to the locations in memory of a computer where different constants are stored. These locations can contain character, integer, real, or any such constants.

**Rules for constructing variable names:**

A variable name is any combination of alphabets, digits or underscores.

- The first character in the variable name must be an alphabet.
- No commas or blanks are allowed within a variable name.
- No special symbol other than an underscore (as in gross\_sal) can be used in a variable name.

**For example:**

si\_int

m\_hra

bassal

pop\_e\_89

The rules for constructing variables remain same for all types of variables: integer variables, real variables, character variables, string variables etc. C compiler is able to distinguish between the variable names, by making it compulsory for you to declare the type of any variable name, which you wish to use in a program. This type declaration is done at the beginning of the program.

**Example:**

```
int si, m_hra;
float basal;
char code;
```

It is a good practice to provide some meaningful variable names.

### Data type:

'Data' is nothing but information. There are two fundamental types of data in C programming - integer and floating point. From these two, we can derive two others - character and double precision. There also exists a data type called void

| Data type | Meaning                        | Size (bytes) |
|-----------|--------------------------------|--------------|
| char      | A character                    | 1            |
| int       | An integer                     | 2            |
| float     | A single precision real number | 4            |
| double    | A double precision real number | 8            |
| void      | Valueless                      | 0            |

The data type **char** is used to store any character belonging to the C character set seen earlier. The data type **int** type is used to store positive or negative integers. A **float** data type stores a single precision floating point (real) number. A **double** data type can store a floating-point value with a greater exactness than a float. The data type void is used to specify an empty set of values.

### **Type Modifiers:**

Short, long, signed, unsigned

Note: signed is default

Variable Definition vs Declaration- Discussed Later (in next pages)

### **Summary of data type and their ranges:**

| Data Type                           | Range                       | Bytes | Format |
|-------------------------------------|-----------------------------|-------|--------|
| Signed char<br>Or<br>Char           | -128 to +127                | 1     | %c     |
| Unsigned char                       | 0 to 255                    | 1     | %c     |
| Int<br>or<br>Signed int             | -32768 to +32767            | 2     | %d     |
| Unsigned int                        | 0 to 65535                  | 2     | %u     |
| Short int<br>or<br>Short signed int | -32768 to +32767            | 2     | %d     |
| Short unsigned int                  | 0 to 65535                  | 2     | %u     |
| Long int<br>or<br>Long signed int   | -2147483648 to + 2147483647 | 4     | %ld    |
| Long unsigned int                   | 0 to 4294967295             | 4     | %lu    |
| Float                               | -3.4e38 to +3.4e38          | 4     | %f     |
| Double                              | -1.7e308 to +1.7e308        | 8     | %lf    |
| Long double                         | -1.7e4932 to 1.7e4932       | 10    | %Lf    |

### **How range is calculated:**

One char variable takes 1 byte=8bits

### **Signed char can store positive as well as negative numbers (+/-)**

|                        |                        |                        |                        |                        |                        |                        |                        |
|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|
| +/-                    | 1                      | 1                      | 1                      | 1                      | 1                      | 1                      | 1                      |
| 8 <sup>th</sup><br>bit | 7 <sup>th</sup><br>bit | 6 <sup>th</sup><br>bit | 5 <sup>th</sup><br>bit | 4 <sup>th</sup><br>bit | 3 <sup>rd</sup><br>bit | 2 <sup>nd</sup><br>bit | 1 <sup>st</sup><br>bit |

8<sup>th</sup> bit is used to store sig(+/-)

Remaining 7 bits are used to store data so 7 bits can represent decimal number:

$$2^7 = 2 * 2 * 2 * 2 * 2 * 2 * 2 = 128$$

-128, -127, -126.....0, 1, 2, 3 .....126, 127

Unsigned char can store only positive numbers

|                 |                 |                 |                 |                 |                 |                 |                 |
|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
| 1               | 1               | 1               | 1               | 1               | 1               | 1               | 1               |
| 8 <sup>th</sup> | 7 <sup>th</sup> | 6 <sup>th</sup> | 5 <sup>th</sup> | 4 <sup>th</sup> | 3 <sup>rd</sup> | 2 <sup>nd</sup> | 1 <sup>st</sup> |
| bit             | bit             | bit             | bit             | bit             | bit             | bit             | bit             |

All 8 bits are used to store data so 8 bits can represent decimal number:

$$2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = 256$$

0, 1, 2, 3.....255

## SYMBOLIC CONSTANT

· When a constant is used at many places in a program, due to some reason if the value of that constant needs to be changed, then change at every statement where that constant occurs in the program –so modification of program becomes difficult.

Symbolic constant is defined as below:

**#define symbolic\_name value**

Example:

**#define FLAG 1**

**#define PI 3.1415**

Here, FLAG and PI are symbolic constants. For better readability, it is advisable to use uppercase character in the naming of symbolic constants. Note that there is no semicolon ‘;’ at the end.

```
#include <stdio.h>
```

```
#define PI 3.14
```

```
void main()
```

```
{    int r=5;
    float area;
    area=PI*r*r;
```

```
}
```

Note: #define is a preprocessor directives like #include. and al preprocessor directive are resolved in preprocessing step.

## Before Operators

Expression= combination of operators and operands

c=a+b

here a, b, and c are operands

and =, + are operators

## Comments in C

| Multi-line Comment                                             | Single-line Comments                                            |
|----------------------------------------------------------------|-----------------------------------------------------------------|
| Starts with /* and ends with */                                | Starts with //                                                  |
| All Words and Statements written between /* and */ are ignored | Statements after the symbol // upto the end of line are ignored |
| Comment ends when */ Occurs                                    | Comment Ends whenever ENTER is Pressed and New Line Starts      |
| e.g /* This is Multiline Comment */                            | e.g // Single line Comment                                      |

```
int main()
{
//single line comment-comments are ignored by compiler
printf("hello world");
return 0;
}
```

```
int main()
{
/*Multi line comment
   Line 1
   Line2
*/
printf("hello world");
return 0;
}
```

### Type Casting:

1. **implicit type conversion**
2. **Explicit type conversion**

Automatic conversion is called implicit type conversion.

```
int x; float f; double d; int i;
x=i      +      i*f      -d
```

```
int c=1, d=2;
float a=5;
printf("%f",a)→ 5.000000
a=c+d;
```

Sometimes we are needed to force the compiler to explicitly convert the value of an expression to a particular data type. This would be clear from the following example:

```
#include<stdio.h>
main( ) {
float a ;
int x = 6, y = 4 ;
a= x/y;
printf ( "Value of a = %f", a ) ;
}
```

And here is the output...

**Value of a = 1.000000**

The answer turns out to be 1.000000 and not 1.5. This is because, 6 and 4 are both integers and hence 6/ 4 yields an integer, 1. This 1 when stored in a is converted to 1.000000. But what if we don't want the quotient to be truncated. One solution is to make either x or y as float. Let us say that some other requirement of the program does not permit us to do this. In such a case what do we do? Use typecasting. The following program illustrates this.

```
#include<stdio.h>
main() {
float a ;
int x = 6, y = 4 ;
a = (float) x / y ;
printf("Value of a = %f", a) ;
}
```

And here is the output...

**Value of a = 1.500000**

This program uses 'typecasting'. This consists of putting a pair of parentheses around the name of the data type. In this program we said,

```
a=(float)x/y;
```

The expression ( float ) causes the variable x to be converted from type int to type float before being used in the division operation.

### Expression, Evaluation of expression

|                                        |                                     |
|----------------------------------------|-------------------------------------|
| $\frac{a^2 + b^2}{2}$                  | $(a*a+b*b)/2$                       |
| $x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$ | $x=(-b+\text{sqrt}(b*b-4*a*c))/2*a$ |
| $\sqrt{s(s-a)(s-b)(s-c)}$              | $\text{Sqrt}(s*(s-a)*(s-b)*(s-c))$  |

```
float a=9, b=12, c=3, x, y, z;
x=a - b / 3 + c * 2 - 1; ➔ 10.000000
y=a - b / (3 + c) * (2 -1) ➔7.000000
```

Ex2:

```
x=a-b/3+c*2-1
x=9-12/3+3*2-1
Step1: x=9-4+3*2-1
Step2: x=9-4+6-1
Step3: x=5+6-1
Step4: x=11-1
Step5: x=10
```

Expression:

Problem1  
 $12*3\%7<<3-2\rightarrow?$   
 (precedence table, modulo operator, bitwise shift left)

Problem2  

```
int a=13, b=25, c, d;
c=a & b;
d=a && b;
printf("\n%d", c);
printf("\n%d", d);
```

## Operators and Expressions

Depending on the function performed, the operators can be classified as:

1. Arithmetic Operators (+, -, \*, \, %)
2. Relational Operators(<, <=, >, >=, ==, !=)
3. Logical Operators(&&, ||, !)
4. Assignment Operators(=, +=, -=, \*=, \=, %=)
5. Increment and Decrement Operators(++ , --)
6. Conditional Operators(?:)
7. Special Operators (comma (,), sizeof)
8. Bitwise Operators(&, |, ^, <<, >>)

In addition to this classification, operators can also be designated as 'unary', 'binary' or 'ternary' depending on whether they operate on one, two, or three operands respectively.

- unary (+3, -78, ++x, x--) → number of operand=1
- binary (a\*b, c/d, a%b) → number of operands=2
- ternary( exp1?exp2:exp3) → number of operands=3

**What is an operator?** It is a symbol, which represents a particular operation that can be performed on some data. The data itself (which can be either a variable or a constant) is called the 'operand'. The operator thus operates on an operand. It's much simple to imagine these through an example. In the expression:

**4 + 2 - 3**

The symbols + and - are operators and the constants 4, 2 and 3 are operands. All these operators are 'binary' operators since they operate on two operands at a time.

### 1- Arithmetic operators:

The arithmetic operators are +, -, \*, / and the modulus operator %.

#### Integer Arithmetic

a=14 and b=4

a-b=10

a+b=18

a\*b=56

a/b=3 (decimal part truncated) –Because both operands of operator '/' are integer

The expression: **x % y** Produces the remainder when x is divided by y, and thus is zero when y divides x exactly.

**14%3=2**

**12%3=0**

**-14%3=-2**

**-14%-3=-2**

**14%-3=2**

Note: modulo operator can be applied only to integer numbers (char, short, int, long)

#### Real Arithmetic

An arithmetic operation involving only real operands is called real arithmetic.

float x;

x=6.0/7.0=.857143

#### Mixed-mode arithmetic:

When one of the operands is real and other is integer, the expression is called mixed mode arithmetic expression will be converted to biggest data type.

15/10.0=1.5

Whereas 15/10=1

|                               |                                                   |
|-------------------------------|---------------------------------------------------|
| Int A;<br>A=5/2→ 2<br>A=6/2→3 | Float B;<br>B=5/2→ 2.000000<br>B=5.0/2.0→2.500000 |
|-------------------------------|---------------------------------------------------|

|  |                                      |
|--|--------------------------------------|
|  | B=5.0/2→2.500000<br>B=5/2.0→2.500000 |
|--|--------------------------------------|

## 2-Relational Operator

These operators are used to compare two operands to see whether they are equal to each other, unequal, or whether one is greater than the other. The operands can be variables, constants or expressions that ultimately get evaluated to a numerical value. Since characters are also represented internally as integers, elements to be compared can be characters as well. C has six relational operators:

| Operator | Meaning                  |
|----------|--------------------------|
| <        | less than                |
| >        | greater than             |
| <=       | less than or equal to    |
| >=       | greater than or equal to |
| ==       | equal to                 |
| !=       | not equal to             |

```
#include<stdio.h>
main( ) {
int a=10, b=20, c=30, d, e;
d = a>b;
e = b<=c;
printf("%d, %d", d, e); (output 0,1)
}
```

### Equality Operators:

#### Operator

==  
!=

#### Example Meaning

x == y x is equal to y  
x != y x is not equal to y

### Relational Operators:

#### Operator

>  
<  
>=  
<=

#### Example Meaning

x > y x is greater than y  
x < y x is less than y  
x >= y x is greater than or equal to y  
x <= y x is less than or equal to y

## 3-Logical Operators:

An expression in C evaluates to either true (non-zero value) or false (zero). In a number of situations it may become necessary to combine the results of evaluation of such expressions. For this purpose, C provides three operators &&, || and !, standing for logical and, logical or, and logical not.

The && operator is usually used in situations where a set of statements are to be executed if two expressions are true. In such cases the two expressions can be connected as shown below:

**(expression 1) && (expression 2)**

If it is enough for either of them to be true then logical or could be made use of:

**(expression 1) || (expression 2)**

The compound expressions formed by using relational and logical operators again evaluate to either true or false.

The not (!) operator reverses the value of the expression it operates on. It makes a true expression false and a false expression true.

Here is an example of the not operator applied to a relational expression.

**!(y<10)**

This means "not y less than 10". We could express the same condition as :

**(y>=10)**

| Opearnd1 | Operand2 | Opearnd1&& Opearnd2 | Opearnd1   Opearnd2 |
|----------|----------|---------------------|---------------------|
| Non zero | Non zero | 1                   | 1                   |

|          |          |   |   |
|----------|----------|---|---|
| Non zero | 0        | 0 | 1 |
| 0        | Non zero | 0 | 1 |
| 0        | 0        | 0 | 0 |

5 && -6 → true → 1

5 && 0 → false → 0

Let a=10, b=20, c=30

a>=5 && b>=10 → true → 1

a+b || b-c → true → 1

#### 4-Assignment Operators

Assignment operators are used to combine the '=' operator with one of the binary arithmetic operators

Let c=9

| Operator | Example | Equivalent Statement | Results |
|----------|---------|----------------------|---------|
| =        | C=7     |                      |         |
| +=       | c += 7  | c = c + 7            | c = 16  |
| -=       | c -= 8  | c = c - 8            | c = 1   |
| *=       | c *= 10 | c = c * 10           | c = 90  |
| /=       | c /= 5  | c = c / 5            | c = 1   |
| %=       | c %= 5  | c = c % 5            | c = 4   |

#### 5-Increment and Decrement Operators:

C offers two special operators ++ and -- called increment and decrement operators, respectively. These are 'unary' operators since they operate on only one operand.

**The operand has to be a variable and not a constant.** Thus, the expression 'a++' is valid whereas '6++' is invalid.

The use of these operators results in incrementing or decrementing the value of the variable by 1. So the expression a++ increments the value in a by 1, and the expression a-- decrements it by 1.

These operators can be used either before or after their operand (i.e. in either prefix or postfix position), so we can have **a++ (postfix)** as well as **++a (prefix)**, and **a-- as well as --a**.

Prefix and postfix operators have the same effect if they are used in an isolated C statement. For example, the effect of the following two statements would be the same.

**a++; ++a;**

However, prefix and postfix operators have different effects when used in association with some other operator in a C statement. For example, if we assume the value of the variable a to be 5, then execution of the statement:

**b = ++a;**

will first increase the value of a to 6 and then assign that new value to b. The effect is exactly the same as that of the following two statements:

**a = a+1; //first increment**

**b = a; //then assignment**

On the other hand, execution of the statement:

**b = a++;**

will first set the value of b to 5 and then increase the value of a to 6.

The effect now is the same as that of the following two statements:

**b = a; //First assignment**

**a = a+1; //Then increment**

```
int a = 9;
printf("%d", a++);
printf("\n");
printf("%d", a);
```

The output would be:

9  
10

The decrement operators are used in a similar way, except, of course, the value of a and b are decreased.

First the prefix decrement: **b = --a;**

which is the same as : **a = a-1; b = a;**

Now the postfix decrement:    b = a--;  
which is the same as :        b =a; a =a-1;

## 6-Conditional Operators:

The conditional operators '?' and ':' are sometimes called ternary operators since they take three operands. Their general form is:

**expression1? expression2: expression3**

What this form says is: "if expression1 is true (that is, if its value is non zero), then the value returned will be expression2, otherwise the value returned will be expression3". Here are a few practical examples of conditional operators.

**int x, y;**

**scanf("%d", &x);**

**y = (x > 5 ? 3 : 4);    OR    x > 5 ? y=3 : y=4;**

This statement will store 3 in y if x is greater than 5, otherwise it will store 4 in y.

x=1?2:3;

x=2

x=0?2:3;

x=3

## 7-Special Operator

### 7-A)    The Comma Operator:

The comma operator (,) permits two different expressions to appear in situations where only one expression would ordinarily be used. The comma operator separates the expressions. The following program illustrates the use of comma operator.

**#include<stdio.h>**

**main(){**

**int a,b,c;**

**c= (a=10, b=20, a+b);**

**printf("%d", c);**

**}** Here, firstly value 10 is assigned to a, followed by 20 assigned to b and then the result of a + b is assigned to c

**a=10, 20, 30;**

**30 will be stored in a**

### 7-B)    Sizeof Operator:

The sizeof operator returns the number of bytes the operand occupies in memory. The operand may be a variable, a constant or a data type qualifier. Consider the following program.

**#include <stdio.h>**

**main(){**

**int sum, char b;**

**printf("%d", sizeof(float));**

**printf("%d", sizeof (sum));**

**printf("%d", sizeof (234L)); //long int**

**printf("%d", sizeof ('A'));**

**printf("%d", sizeof (b));**

**}**

Here the first printf( ) would print out 4 since a float is always 4 bytes long. With this reasoning, the next four printf( ) statements would output 2, 4, 2 and 1.

## 8- Bitwise operator (Not Covered)

The smallest element in memory on which we are able to operate as yet is a byte.

C provides six operators for bit manipulation. These may only be applied to integral operands, i.e., char, short, int and long, whether signed or unsigned.

**&**    bitwise AND

**|**    bitwise inclusive OR

**^**    bitwise exclusive OR

**<<**   left shift

**>>**   right shift

**~**    one's complement (unary)

### Bitwise Operators – truth table

| A | b | a&b | a b | a^b | ~a |
|---|---|-----|-----|-----|----|
| 0 | 0 | 0   | 0   | 0   | 1  |
| 0 | 1 | 0   | 1   | 1   | 1  |
| 1 | 0 | 0   | 1   | 1   | 0  |
| 1 | 1 | 1   | 1   | 0   | 0  |

```

11010011
&
10001100
-----
10000000

```

```

11010011
|
10001100
-----
11011111

```

```

11010011
^
10001100
-----
01011111

```

```

~11010011
-----
00101100

```

```

11010011>>3
-----
00011010

```

```

11010011<<3
-----
10011000

```

Let

unsigned int X=13; unsigned int Y=25;

|          |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|----------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| X (13)   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| Y (25)   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| X&Y (9)  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| X Y (29) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 |
| X^Y (20) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |

### Imporant-Short circuit and Logical Operators (&& , ||) in C

Short circuiting in C is when a logical operator doesn't evaluate all its arguments.

Take for example and &&, it's pretty obvious that 0 && WhoCares is going to be false no matter what WhoCares is. Because of this, C just skips evaluating WhoCares. Same goes for 1 || WhoCares, it'll always be true. Because of this, we can write code like CanFireMissiles && FireMissiles()

This way we avoid doing some potentially impossible operation. If we can't fire the missiles we certainly don't want to try to.

This plays out in lots of other useful ways to avoid unnecessary computing

isFileReady() || getFileReady()

This avoids doing extra work if we don't need to.

### Avoiding undesired side effects of the second argument

Usual example, using a C-based language:

```

int denom = 0;
if (denom != 0 && num / denom)
{
    ... // ensures that calculating num/denom never results in divide-by-zero error
}

```

Consider the following example:

```

int a = 0;
if (a != 0 && myfunc(b))
{
    do_something();
}

```

In this example, short-circuit evaluation guarantees that myfunc(b) is never called. This is because a != 0 evaluates to *false*. This feature permits two useful programming constructs.

Consider following expression:

$X = A + B * C$

$X = A * B * C / D * E$

$X = A * B * (C / D) * E$

## C Operator Precedence Table

This page lists C operators in order of *precedence* (highest to lowest). Their *associativity* indicates in what order operators of equal precedence in an expression are applied.

| Operator                                          | Description                                                                                                                                                                                                                       | Associativity |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| ( )<br>[ ]<br>.<br>-><br>++ --                    | Parentheses (function call) (see Note 1)<br>Brackets (array subscript)<br>Member selection via object name<br>Member selection via pointer<br>Postfix increment/decrement (see Note 2)                                            | left-to-right |
| ++ --<br>+ -<br>! ~<br>(type)<br>*<br>&<br>sizeof | Prefix increment/decrement<br>Unary plus/minus<br>Logical negation/bitwise complement<br>Cast (convert value to temporary value of type)<br>Dereference<br>Address (of operand)<br>Determine size in bytes on this implementation | right-to-left |
| * / %                                             | Multiplication/division/modulus                                                                                                                                                                                                   | left-to-right |
| + -                                               | Addition/subtraction                                                                                                                                                                                                              | left-to-right |
| << >>                                             | Bitwise shift left, Bitwise shift right                                                                                                                                                                                           | left-to-right |
| < <=<br>> >=                                      | Relational less than/less than or equal to<br>Relational greater than/greater than or equal to                                                                                                                                    | left-to-right |
| == !=                                             | Relational is equal to/is not equal to                                                                                                                                                                                            | left-to-right |
| &                                                 | Bitwise AND                                                                                                                                                                                                                       | left-to-right |
| ^                                                 | Bitwise exclusive OR                                                                                                                                                                                                              | left-to-right |
|                                                   | Bitwise inclusive OR                                                                                                                                                                                                              | left-to-right |
| &&                                                | Logical AND                                                                                                                                                                                                                       | left-to-right |
|                                                   | Logical OR                                                                                                                                                                                                                        | left-to-right |
| ? :                                               | Ternary conditional                                                                                                                                                                                                               | right-to-left |
| =<br>+= -=<br>*= /=<br>%= &=<br>^=  =<br><<= >>=  | Assignment<br>Addition/subtraction assignment<br>Multiplication/division assignment<br>Modulus/bitwise AND assignment<br>Bitwise exclusive/inclusive OR assignment<br>Bitwise shift left/right assignment                         | right-to-left |
| ,                                                 | Comma (separate expressions)                                                                                                                                                                                                      | left-to-right |

**Note 1:**

Parentheses are also used to group sub-expressions to force a different precedence; such parenthetical expressions can be nested and are evaluated from inner to outer.

**Note 2:**

Postfix increment/decrement have high precedence, but the actual increment or decrement of the operand is delayed (to be accomplished sometime before the statement completes execution). So in the statement  $y = x * z++$ ; the current value of  $z$  is used to evaluate the expression (*i.e.*,  $z++$  evaluates to  $z$ ) and  $z$  only incremented after all else is done. See [postinc.c](#) for another example.

### Formatted Input/ Output Function (Not covered)

Explain working of printf and scanf function.

#### Formatted input

scanf("control string",arg1, arg2,.....argn);

%wd

w is field width of the number to be read

scanf("%2d%5d", &num1, &num2);

if input=>50 31426        then num1=50 num2=31426

but if input=>31426 50

then num1=31 because of %2d

and num2=426 (unread part of 31426)

the value 50 unread and will be assigned to first variable in next scanf if any.

#### Formatted Output

printf("control string",arg1, arg2,.....argn);

##### Integers formatting

|                     |   |   |   |   |
|---------------------|---|---|---|---|
| printf("%d", 9877); | 9 | 8 | 7 | 7 |
|---------------------|---|---|---|---|

|                      |  |   |   |   |   |
|----------------------|--|---|---|---|---|
| printf("%5d", 9877); |  | 9 | 8 | 7 | 7 |
|----------------------|--|---|---|---|---|

##### Left Justification and Right Justification

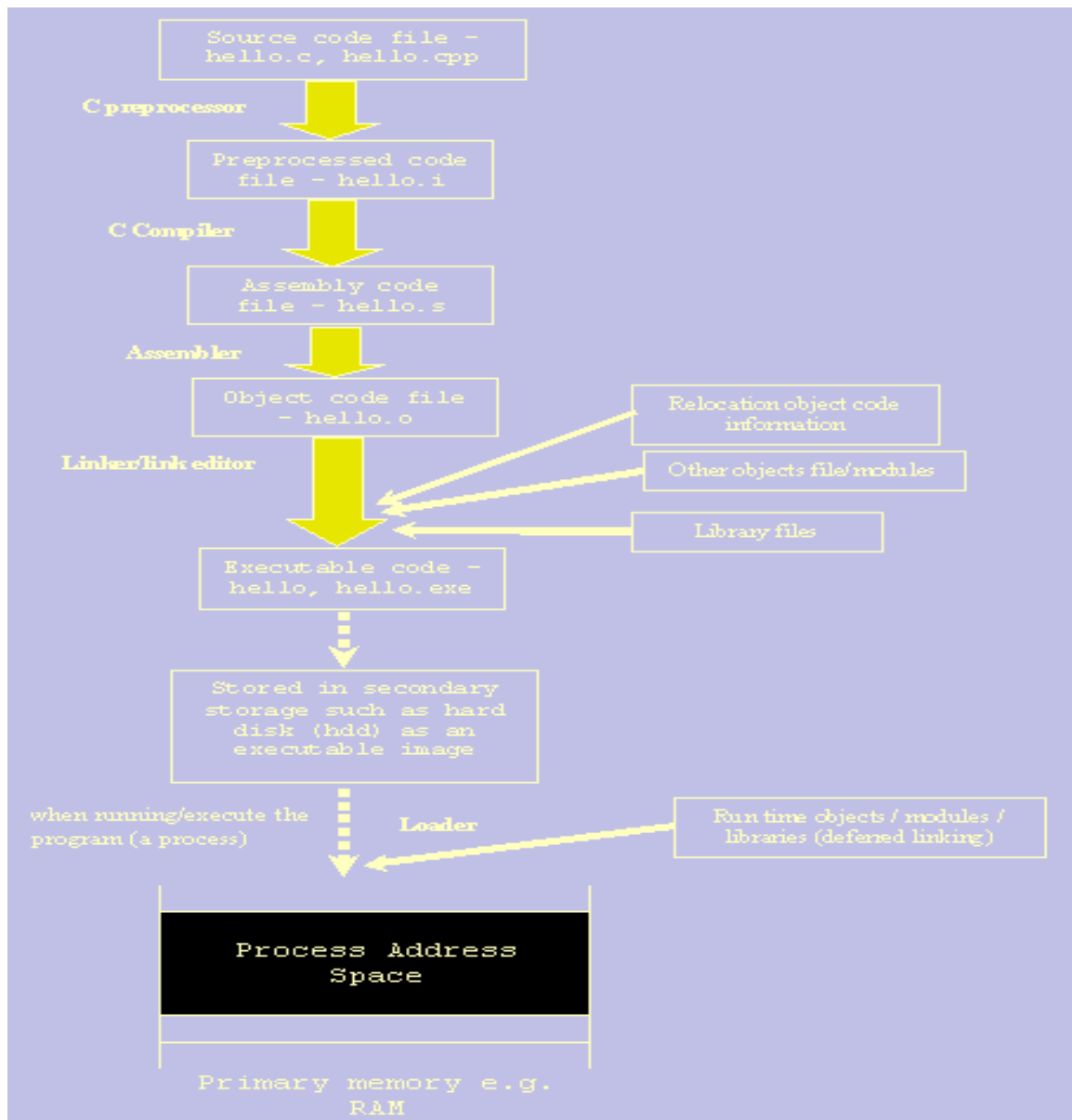
*Floating Point Numbers*

float x = 98.7654;

|                    |   |   |   |   |   |   |   |
|--------------------|---|---|---|---|---|---|---|
| printf("%.4f", x); | 9 | 8 | . | 7 | 6 | 5 | 4 |
|--------------------|---|---|---|---|---|---|---|

|                    |  |  |   |   |   |   |   |
|--------------------|--|--|---|---|---|---|---|
| printf("%.2f", x); |  |  | 9 | 8 | . | 7 | 7 |
|--------------------|--|--|---|---|---|---|---|

## Compilation steps / Life of a C Program / Text and Binary files in context with different phases in life of a C program



### Variable Definition vs Declaration (Ref- <http://www-ee.eng.hawaii.edu>)

(This can be understood well after studying storage class mainly extern)

For automatic and register variables, there is no difference between definition and declaration. The process of declaring an automatic or a register variable defines the variable name and allocates appropriate memory.

However, for external variables, these two operations may occur independently. *This is important because memory for a variable must be allocated only once, to ensure that access to the variable always refers to the same cell. Thus, all variables must be defined once and only once.*

A declaration statement may also be used to simply *declare* a variable name with the extern storage class specifier at the beginning of the declaration. Such a declaration specifies that the variable is *defined* elsewhere, i.e. memory for this variable is allocated in another file. Thus, access to an external variable in a file other than the one in which it is *defined* is possible if it is *declared* with the keyword extern; no new memory is allocated. Such a declaration tells the compiler that the variable is defined elsewhere, and the code is compiled with the external variable left unresolved. The reference to the external variable is resolved during the linking process.

Here are some examples of *declarations* of external variables that are not *definitions*:

**extern int stkptr;**

These declarations tell the compiler that the variable stkptr are defined elsewhere, usually in some other file. If the keyword extern were omitted, the variables would be considered to be new ones and memory would be allocated for them. Remember, access to the same external variable defined in another file is possible only if the keyword extern is used in the

### Constant vs Symbolic Constant

#### Or Comparison between constant and macros

#### Or Constant using “const” keyword vs Constant using “#define”

Macro-defined constants are replaced by the preprocessor. Constant 'variables' are managed just like regular variables.

For example, the following code:

```
#define A 8
```

```
int b = A + 10;
```

Would appear to the actual compiler as

```
int b = 8 + 10;
```

However, this code:

```
const int A = 8;
```

```
int b = A + 10;
```

Would appear as:

```
const int A = 8;
```

```
int b = A + 10;
```

;)

In practice, the main thing that changes is scope: constant variables obey the same scoping rules as standard variables in C, meaning that they can be restricted, or possibly redefined, within a specific block, without it leaking out - it's similar to the local vs. global variables situation.

### Macro function vs Inline function

A Macro in C is simply text that is expanded *before* the compiler processes the source code. The inline keyword is used as a hint to the compiler that the function can be placed inline without the need for a call-stack to be set up.

So, for example, lets say that you got the following two snippets of code (First the macro and then the inline function):

```
#define MAX(x,y) (x > y ? x : y)
```

and

```
inline int max(int x, int y) { return x > y ? x : y; }
```

When the preprocessor finds the following call in your code:

```
int highest = MAX (var_1, var_2);
```

it replaces it with

```
int highest = (var_1 > var_2 ? var_1 : var_2);
```

The above is what the compiler eventually gets during the compilation process, hence the snippet defined by MAX(x,y) is the *replacement* for MAX(var\_1, var\_2). When the compiler finds the function call

```
int highest = max (var_1, var_2);
```

Then the function "max" gets called. You must assume that it gets called the normal way, because the compiler is free to ignore your "inline" hint and make calls to the function go through the normal call-stack rather than simply placing the code for the function in the place it is encountered.

One last caveat with macros: because it is all text replacement and not code replacement, if you do something like this:

```
int highest = MAX (v1++, v2++);
```

the preprocessor will expand that to:

```
int highest = (v1++ > v2++ ? v1++ : v2++);
```

**inline** is not same as **macro** definition, as macro is preprocessed but inline is just the indication to the compiler to put the whole body of the function where it is called. Putting inline keyword before any function definition is just the indication or request, compiler is free to choose whether it's going to make that function inline or leave it as normal

## UNIT 3

### If Else Switch Loops Jump Statement

#### Difference between performance of if else and switch, Advantages of if else and switch over each other

This is the general syntax of if-else ladder:

```
1. if (condition1) { //Body of if }
2. else if (condition2) { //Body of if }
3. else if (condition3) { //Body of if }
And this is the general syntax for switch:
switch ( variable )
1. {
2. case <variable value1>: //Do Something
3.     break;
4. case <variable value2>: //Do Something
5.     break;
6. default: //Do Something
7.     break;
8. }
```

The if-else ladder is of type strict condition check, while switch is of type jump value catching.

#### Switch vs If-else Answer1

##### Limitations of switch over if-else ladder:

The variable expression are also not allowed in cases.

**case i+2:** is not allowed in switch, but is valid on if-else.

- You cannot test a flat expression using switch.
- You cannot use similar expressions for multiple cases. For instance below **switch statement in c** is illegal:

```
1. switch (expression)
2. {
3.     case 5: //...
4.         break;
5.     case 2+3: //...
6.         break;
7. }
```

- **switch statement** must know the value inside it during compilation. But in c programming most of the time, it is required depending on the user input. In such cases, you cannot use switch.

##### Advantages of switch over if-else ladder:

- In some situations a switch statement works much faster than equivalent if-else ladder.  
*It is because compiler generates a jump table for a switch during compilation. Consequently, during execution, instead of checking which case is satisfied, it only decides which case has to be executed.*
- It is more readable and in compare to if-else statements.
- It is more manageable for having higher level of indentation than if. For instance check below two source codes (solving same problem one using if where the other using switch) to check error messages.

##### Where to use switch over if-else ladder:

- If there are large number of compares for a condition in your program, use switch over if-else ladder.
- For more complex comparisons.

##### Where to use if-else ladder over switch:

In case of simple and few compares, if-else executes faster and easy write. Thus as per program's requirement, a programmer should decide himself where to use which one condition control.

#### Switch vs If-else Answer2

**if else** and **switch case** statements are nearly same. We can do conditional checking using these. But in one position these two have a little variations. We can't operate **float** or **double** values using switch case, but we can do using if else statements. This is the difference. but **switch case is more efficient** than if else statement. **Debugging is more easy in switch case.**

The switch statement is easy to edit as it has created the separate cases for different statements whereas, in nested if-else statements it become difficult to identify the statements to be edited.

##### Key Differences Between if-else and switch

1. Expression inside if statement decide whether to execute the statements inside if block or under else block. On the other hand, expressions inside switch statement decide which case to execute.
2. You can have multiple if statement for multiple choice of statements. In switch you only have one expression for the multiple choices.

3. If-else statement checks for equality as well as for logical expression. On the other hand, switch checks only for equality.
4. The if statement evaluates integer, character, pointer or floating-point type or Boolean type. On the other hand, switch statement evaluates only character or a integer data type.
5. Sequence of execution is like either statement under if block will execute or statements under else block statement will execute. On the other hand the expression in switch statement decide which case to execute and if you do not apply a break statement after each case it will execute till the end of switch statement.
6. If expression inside if turn out to be false, statement inside else block will be executed. If expression inside switch statement turn out to be false then default statements is executed.
7. It is difficult to edit if-else statements as it is tedious to trace where the correction is required. On the other hand it is easy to edit switch statements as they are easy to trace.

### Loops in C (Loops – ‘for’ loops, ‘while’ loops, ‘do while’ loops)

(Ref- <https://www.tutorialspoint.com>, <https://www.programiz.com>)

You may encounter situations, when a block of code needs to be executed several number of times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times.

Loops are used in programming to repeat a specific block until some end condition is met. There are three loops in C programming:

1. for loop
2. while loop
3. do...while loop

#### For Loop

The syntax of for loop is:

```
for (initializationStatement; testExpression; updateStatement)
{
    // codes
}
```

#### How for loop works?

The initialization statement is executed only once. Then, the test expression is evaluated. If the test expression is false (0), for loop is terminated. But if the test expression is true (nonzero), codes inside the body of for loop is executed and the update expression is updated. This process repeats until the test expression is false.

The for loop is commonly used when the number of iterations is known.

Example: for loop

// Program to calculate the sum of first n natural numbers

// Positive integers 1,2,3...n are known as natural numbers

#include <stdio.h>

int main()

```
{
    int num, count, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &num);
    // for loop terminates when n is less than count
    for(count = 1; count <= num; ++count)
    {
        sum += count;
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Output:

Enter a positive integer: 10

Sum = 55

#### while loop

The syntax of a while loop is:

```
while (testExpression)
{
    //codes
}
```

where, testExpression checks the condition is true or false before each loop.

### How while loop works?

The while loop evaluates the test expression. If the test expression is true (nonzero), codes inside the body of while loop are executed. The test expression is evaluated again. The process goes on until the test expression is false. When the test expression is false, the while loop is terminated.

### do...while loop

The do...while loop is similar to the while loop with one important difference. The body of do...while loop is executed once, before checking the test expression. Hence, the do...while loop is executed at least once.

### do...while loop Syntax

```
do
{
    // codes
}
while (testExpression);
```

### How do...while loop works?

The code block (loop body) inside the braces is executed once. Then, the test expression is evaluated. If the test expression is true, the loop body is executed again. This process goes on until the test expression is evaluated to 0 (false). When the test expression is false (nonzero), the do...while loop is terminated.

## Entry and Exit Controlled Loop in C (ref- <http://www.includehelp.com>)

Loops are the technique to repeat set of statements until given condition is true. C programming language has three types of loops - **1) while loop**, **2) do while loop** and **3) for loop**.

These loops controlled either at entry level or at exit level hence loops can be controlled two ways

1. Entry Controlled Loop
2. Exit Controlled Loop
- 3.

### Entry Controlled Loop

Loop, where test condition is checked before entering the loop body, known as **Entry Controlled Loop**.

**Example:** while loop, for loop

### Exit Controlled Loop

Loop, where test condition is checked after executing the loop body, known as **Exit Controlled Loop**.

**Example:** do while loop

Consider the code snippets

#### Using while loop

```
1 int count=100;
2 while(count<50)
3 printf("%d",count++);
```

#### Using for loop

```
1 int count;
2 for(count=100; count<50; count++)
3 printf("%d",count);
```

In both code snippets **value of count is 100** and condition is **count<50**, which will check first, hence there is no output.

#### Using do while loop

```
1 int count=100;
2 do
3 {
4 printf("%d",count++);
5 }while(count<50);
```

In this code snippet **value of count is 100** and test condition is **count<50** which is false yet loop body will be executed first then condition will be checked after that. Hence output of this program will be 100.

### Differences Table

| Entry Controlled Loop                                                 | Exit Controlled Loop                                             |
|-----------------------------------------------------------------------|------------------------------------------------------------------|
| Test condition is checked first, and then loop body will be executed. | Loop body will be executed first, and then condition is checked. |
| If Test condition is false, loop body will not be executed.           | If Test condition is false, loop body will be executed once.     |

|                                                                                                          |                                                                                                          |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| for loop and while loop are the examples of Entry Controlled Loop.                                       | do while loop is the example of Exit controlled loop.                                                    |
| Entry Controlled Loops are used when checking of test condition is mandatory before executing loop body. | Exit Controlled Loop is used when checking of test condition is mandatory after executing the loop body. |

### Jumping Statements in C | goto, break, continue, return

Jumping statements are used to transfer the program's control from one location to another; these are set of keywords which are responsible to transfer program's control within the same block or from one function to another.

**There are four jumping statements in C language:**

- goto statement
- return statement
- break statement
- continue statement

#### Break & continue statement

Break is used to terminate the looping (exit from the loop) while continue is used to transfer the program's control at the beginning of the loop.

#### Consider the example:

Here loop counter condition from 1 to 10 and we will use break and continue when counter's value is 7, let's see what will be the output.

```

1  #include <stdio.h>
2  int main()
3  {
4      int cnt;
5      for(cnt=1; cnt<=10; cnt++)
6      {
7          printf("%d ",cnt);
8          if(cnt==7)
9              break;
10         printf("OK\n");
11     }
12     printf("\nAfter the loop cnt= %d",cnt);
13     return 0;
14 }
15
```

#### Output

```

1 OK
2 OK
3 OK
4 OK
5 OK
6 OK
7
After the loop cnt= 7
```

When cnt==7, break will transfer the program's control just after the loop statement ( after the closing brace "}" ).

```

1  #include <stdio.h>
2  int main()
3  {
4      int cnt;
5
6      for(cnt=1; cnt<=10; cnt++)
7      {
8          printf("\n%d ",cnt);
9          if(cnt==7)
10             continue;
11         printf("OK");
12     }
13     printf("\nAfter the loop cnt= %d",cnt);
14     return 0;
15 }
```

#### Output

```

1 OK
2 OK
3 OK
4 OK
5 OK
6 OK
7
8 OK
9 OK
10 OK
After the loop cnt= 11
```

When `cnt==7`, `continue` transfers the program's control at the starting of the loop to test condition again hence After "7" "OK" didn't print.

### **goto statement (not part of syllabus)**

`goto` is a jumping statement in c language, which transfer the program's control from one statement to another statement (where label is defined). `goto` can transfer the program's within the same block and there must a label, where you want to transfer program's control.

Defining a label

Label is defined following by the given syntax

label\_name:

Note:label\_name should be a valid identifier name., : (colon) should be used after the label\_name.

Transferring the control using 'goto'

Program's control can be transfer following by the given syntax

`goto label_name;`

### **Return statement (not part of syllabus)**

`return` statement is used to transfer program's control from called function to calling function, it's secondary task is to carry value from called function to calling function.

If function's return type is `void`, you can use `return` only to return program's control to the calling function, and if there is a return type you will have to return same type of value to the calling function.

```
1      // function with no return type
2      void myFun(void)
3      {
4          printf("Hello");
5          return;
6          printf("Hi");
7      }
8      // will not print Hi
9
10     // function with int return type – to return addition of two numbers
11     int add(int a , int b)
12     {
13         int sum=0;
14         sum=a+b;
15         return sum;
16     }
17     // will transfer program's control to calling function with value of sum.
```

## UNIT 4-ARRAY

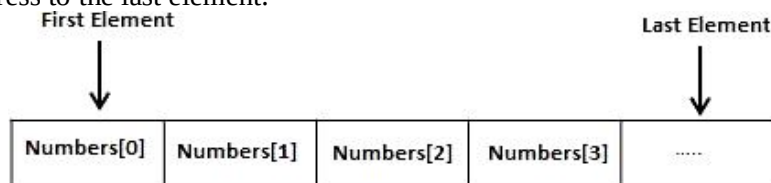
### Definition of Array:

An array is a collection of homogeneous elements or an array is a collective name given to a group of 'similar quantities'. These similar quantities could be percentage marks of 100 students, or salaries of 300 employees, or ages of 50 employees. What is important is that the quantities must be 'similar'. Each member in the group is referred to by its position in the group.

Similar elements of array could be all ints, or all float, or all chars etc. Usually, the array of characters is called a 'string', whereas an array of ints or floats is called simply an array.

Whereas ordinary variables are capable of holding only one value at a time.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



Declaration of 1D Array:

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows:

```
datatype arrayName [ arraySize ];
```

This is called a single-dimensional array. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type. For example, to declare a 10-element array called **marks** of type **int**, use this statement:

```
int marks[10];
```

|          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| Marks[0] | Marks[1] | Marks[2] | Marks[3] | Marks[4] | Marks[5] | Marks[6] | Marks[7] | Marks[8] | Marks[9] |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|

If an integer takes 4 bytes (32 bits) then size occupied by **marks** array will be 40 bytes.

### Initialization of 1D array

- Compile time initialization**

Fully initialization:

```
int num[5] = {2,8,7,6,0};
```

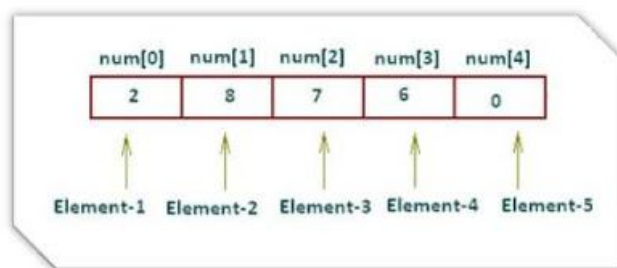
This is also correct:

```
int num[] = {2,8,7,6,0};
```

```
int num[5] = {0}; // all elements of
array will initialize with zero
```

we can initialize an array Partially :

```
int num[5] = {2,8}; // remaining elements will be initialized with zero
```



- Run time initialization**

Using scan function, we can store elements of array at run time.

### Declaration of 2D Array:

```
int abc[5][4];
```

### Initialization of 2D array:

```
int arr[3][4] =
{0,1,2,3,4,5,6,7,8,9,10,11}; //Storage take
place in sequential order
```

|       | Column 0 | Column 1 | Column 2 | Column 3 |
|-------|----------|----------|----------|----------|
| Row 0 | a[0][0]  | a[0][1]  | a[0][2]  | a[0][3]  |
| Row 1 | a[1][0]  | a[1][1]  | a[1][2]  | a[1][3]  |
| Row 2 | a[2][0]  | a[2][1]  | a[2][2]  | a[2][3]  |

```
int arr[2][4] = {
    {10, 11, 12, 13},
    {14, 15, 16, 17}
};
```

|    |    |    |    |
|----|----|----|----|
| 10 | 11 | 12 | 13 |
| 14 | 15 | 16 | 17 |

|   |   |   |
|---|---|---|
| 1 | 1 | 0 |
| 2 | 2 | 2 |
| 3 | 0 | 0 |

```
int arr[3][3] = { { 1,1}, {2,2,2},{3}};
```

```
int abc[2][2] = {1, 2, 3,4 } /* Valid */
```

```
int abc[ ][2] = {1, 2, 3,4 } /* Valid , first dimension can be empty*/
```

```
int abc[ ][ ] = {1, 2, 3,4 } /* Invalid declaration – you must specify second dimension*/
```

```
int abc[2][ ] = {1, 2, 3,4 } } /* Invalid declaration – you must specify second dimension*/
```

## What are Advantages and Disadvantages of arrays?

### Advantages:

1. It is used to represent multiple data items of same type by using only single name.
2. It can be used to implement other data structures like linked lists, stacks, queues, trees, graphs etc.
3. 2D arrays are used to represent matrices.

### Disadvantages:

1. We must know in advance that how many elements are to be stored in array.
2. Array is static structure. It means that array is of fixed size. The memory which is allocated to array cannot be increased or reduced.
3. Since array is of fixed size, if we allocate more memory than requirement then the memory space will be wasted. And if we allocate less memory than requirement, then it will create problem.
4. The elements of array are stored in consecutive memory locations. So insertions and deletions are very difficult and time consuming.

## Array Bounds Checking in C

Most languages (Java, C#, Python, Javascript) prevent the programmer from going past the end of an array. This process, performed at runtime by the language implementation, is frequently called *Bounds Checking*.

*What if programmer accidentally accesses any index of array which is out of bound?*

**Unfortunately in C there is no way for the programmer to determine the size of an array at runtime** and so it becomes necessary for the programmer to keep track of the length of the array.

Array Bounds checking is not available in C language. C doesn't provide any specification which deal with problem of accessing invalid index.

As per ISO C standard it is called **Undefined Behavior**.

An undefined behavior (UB) is a result of executing computer code whose behavior is not prescribed by the language specification to which the code can adhere to, for the current state of the program (e.g. memory). This generally happens when the translator of the source code makes certain assumptions, but these assumptions are not satisfied during execution.

### Examples of Undefined Behavior while accessing array out of bounds

1. **Access non allocated location of memory:** The program can access some piece of memory which is owned by it.  

```
// Program to demonstrate // accessing array out of bounds
#include <stdio.h>
int main()
```

```

{
    int arr[] = {1,2,3,4,5};
    printf("arr [0] is %d\n", arr[0]);

    // arr[10] is out of bound
    printf("arr[10] is %d\n", arr[10]);
    return 0;
}

```

Output :

arr [0] is 1

arr[10] is -1786647872

It can be observed here, that arr[10] is accessing a memory location containing a garbage value.

2. **Segmentation fault:** The program can access some piece of memory which is not owned by it, which can cause crashing of program such as **segmentation fault**.

// Program to demonstrate // accessing array out of bounds

```
#include <stdio.h>
```

```
int main()
```

```

{
    int arr[] = {1,2,3,4,5};
    printf("arr [0] is %d\n",arr[0]);
    printf("arr[10] is %d\n",arr[10]);

```

```
    // allocation memory to out of bound
```

```
    // element
```

```
    arr[10] = 11;
```

```
    printf("arr[10] is %d\n",arr[10]);
```

```
    return 0;
```

```
}
```

Output :

Runtime Error : Segmentation Fault (SIGSEGV)

Core Dump/Segmentation fault is a specific kind of error caused by accessing memory that “does not belong to you.”

#### Accessing out of array index bounds:

// C++ program to demonstrate segmentation

// fault when array out of bound is accessed.

```
int main()
```

```
{
```

```
    int arr[2];
```

```
    arr[3] = 10; // Accessing out of bound
```

```
    return 0;
```

```
}
```

Output:

Abnormal termination of program.

## UNII 5-PART A-FUNCTIONS

### Definition of Function:

A **function** is a group of statements that together perform a task. Every **C** program has at least one **function**, which is `main()`, and all programs can define additional **functions**. We can divide up your code into separate **functions**. A function is known with various names like a method or a sub-routine or a procedure, etc.

### User defined vs Predefined function

The C standard library provides numerous built-in functions that our program can call, these functions are called predefined functions or inbuilt functions or library functions. The prototype declarations of in-built functions are stored in header files.

Library functions are limited. We can define additional **functions to achieve some task. The function that is defined by the user is called User defined function.** The prototype declarations of user defined function is done by the user.

The **return** statement is the mechanism for returning a value from the called function to its caller.

Argument **declaration** indicates parameter; the function receives from the called function

### What is the need of “User Defined Functions”:

- Some time same operation to be done repeatedly in a program.
- Just using main Large, Complex and Difficult to Debug.
- Library functions are limited.

### Advantages of Functions:

- Top down Modular Programming
- Smaller programs
- Bugs limited to an area
- Common function reused by others

### What are Function definition, Function declaration, and Function call?

**Defining a Function:** The general form of a function definition in C programming language is as follows:

```
return_type function_name( parameter list )
{
    body of the function
}
```

A function definition in C programming language consists of a function header and a function body. Here are all the parts of a function:

- **Return Type:** A function may return a value. The **return\_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the **return\_type** is the keyword **void**.
- **Function Name:** This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters:** A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body:** The function body contains a collection of statements that define what the function does.

### Function Declarations:

A function **declaration** tells the compiler about a function's name, return type, and parameters. Or we can say a function **declaration** tells the compiler about a function name and how to call the function.

A function declaration has the following parts:

```
return_type function_name( parameter list );
```

### Calling a Function:

While creating a C function, we give a definition of what the function has to do. To use a function, you will have to **call that function** to perform the defined task.

When a program calls a function, program control is transferred to the called function. A called function performs defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns program control back to the main program.

To call a function, you simply need to pass the required parameters along with function name, and if function returns a value, then you can store returned value

### Function Arguments: (Formal arguments vs Actual arguments)

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.

The formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit. While we are passing argument(s) in function call these are called actual argument(s).

|                                                                                                  |                                                                                       |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <pre>void sum(int,int); int main() { int a=5,b=6; sum(a,b); //actual arguments return 0; }</pre> | <pre>void sum(int x, int y)//formal arguments { printf("The ans is %d", x+y); }</pre> |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|

While calling a function, there are two ways that arguments can be passed to a function:

| Call Type         | Description                                                                                                                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Call by value     | This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.                                        |
| Call by reference | This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument. |

### Call by Value vs Call by Reference

|                                     |                                         |
|-------------------------------------|-----------------------------------------|
| <b>a. Value Passed</b>              | <b>Address Passed</b>                   |
| <b>b. Value unaltered</b>           | <b>Alteration Reflected back</b>        |
| <b>c. Different Memory Location</b> | <b>Same Memory Location</b>             |
| <b>d. Data is safe</b>              | <b>Data can be accidentally altered</b> |

### Different categories of function:

1. With return type and with argument.
2. With return type and without argument.
3. Without return type and with argument.
4. Without return type and without argument.

## UNII 5-PART B-RECURSION

### Recursion vs Iteration

| RECURSION                                                                                                                            | ITERATIONS                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Recursive function – is a function that is partially defined by itself                                                               | Iterative Instructions –are loop based repetitions of a process                       |
| Recursion Uses selection structure                                                                                                   | Iteration uses repetition structure                                                   |
| Infinite recursion occurs if the recursion step does not reduce the problem in a manner that converges on some condition.(base case) | An infinite loop occurs with iteration if the loop-condition test never becomes false |
| Recursion terminates when a base case is recognized                                                                                  | Iteration terminates when the loop-condition fails                                    |
| Recursion is usually slower then iteration due to overhead of maintaining stack                                                      | Iteration does not use stack so it's faster than recursion                            |
| Recursion uses more memory than iteration                                                                                            | Iteration consume less memory                                                         |
| Infinite recursion can crash the system                                                                                              | infinite looping uses CPU cycles repeatedly                                           |
| Recursion makes code smaller                                                                                                         | Iteration makes code longer                                                           |

#### Types of recursion:

**Head Recursion-** In head recursion, the recursive call, when it happens, comes before other processing in the function (think of it happening at the top, or head, of the function).

```

void tail(int n)      void head(int n)
{
    if(n == 1)        {
        return;        if(n == 0)
    else              return;
        Printf("Hello");
        tail(n-1);      head(n-1);
    }                  Printf("Hello");
}                      }

```

**Tail Recursion-** it's the opposite—the processing occurs before the recursive call.

**Other:** Linear recursion, Nested recursion, Binary recursion

## UNIT 5-PART C-STORAGE CLASS

|                                         |                                                                                                             |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Scope</b>                            | Region or Part of Program in which Variable is accessible                                                   |
| <b>Longevity/<br/>Lifetime/Lifespan</b> | it refers to the period during which a variables retains a given value during execution of a program(alive) |

Local (internal) variables: are those which are declared within a particular function.

Global (external) variables: are those which are declared outside any function.

#### Definition of Storage Class:

A **storage class** defines the scope (visibility) and life time of variables and/or functions within a C Program. There are following **storage classes** which can be used in a C Program:

| Storage class | Default Value | Scope                                                                    | Lifetime                                                   | Storage Location                                                       |
|---------------|---------------|--------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------|
| Automatic     | Garbage       | Local to the function or block.                                          | Exists as long as Control remains in the function or block | Main Memory                                                            |
| External      | Zero          | Global / File Scope                                                      | Active and alive throughout the program                    | Main Memory                                                            |
| Static        | Zero          | <b>Internal static-</b> Like Auto<br><b>External static-</b> Like global | Until the end of program                                   | Main Memory                                                            |
| Register      | Garbage       | Local to the function or block.                                          | Exists as long as Control remains in the block             | CPU Register (In case of no space in CPU Register then in Main memory) |

#### Automatic variables

- Are declared inside a function in which they are to be utilized.
- Are declared using a keyword *auto*. eg. *auto int number;*
- Are created when the function is called and destroyed automatically when the function is exited.
- These variables are therefore private (local) to the function in which they are declared.
- Variables declared inside a function without storage class specification is, by default, an automatic variable.

#### Some more on Automatic Variables:

- During recursion, the nested variables are unique **auto** variables.
- Automatic variables can also be defined within blocks. In that case they are meaningful only inside the blocks where they are declared.
- If automatic variables are not initialized they will contain garbage

#### External Variables

- These variables are declared outside any function.
- These variables are active and alive throughout the entire program.
- Also known as global variables and default value is zero.
- Unlike local variables they are accessed by any function in the program.
- In case local variable and global variable have the same name, the local variable will have precedence over the global one.
- Sometimes the keyword **extern** used to declare these variables.
- It is visible only from the point of declaration to the end of the program.

#### Static Variables

- The value of static variables persists until the end of the program.
- It is declared using the keyword static like  

```
static int x;
static float y;
```
- It may be of external or internal type depending on the place of their declaration.
- **Static variables are initialized only once, when the program is compiled.**

#### Register Variable

- These variables are stored in one of the machine's register and are declared using register keyword.  
eg. *register int count;*
- Since register access are much faster than a memory access keeping frequently accessed variables in the register lead to faster execution of program.
- Since only few variables can be placed in the register, it is important to carefully select the variables for this purpose. However, C will automatically convert register variables into non-register variables once the limit is reached.

Note: Don't try to declare a global variable as register. Because the register will be occupied during the lifetime of the program.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
auto int i = 10;
```

```
printf("%d\n",function1());
```

```
printf("%d\n",function2());
```

```
printf("%d\n",function1());
```

```
printf("%d\n",function2());
```

```
printf("%d\n",i);
```

```
}
```

```
int function1()
```

```
{
```

```
int j = 0;
```

```

j++;
return(j);
}
int function2()
{
static int i = 100;
i+=1;
return(i);
}

```

### Output

```

1
101
1
102
10

```

## UNIT 5-PART D-MEMORY LAYOUT OF C PROGRAM

In practical words, when we run any C-program, its executable image is loaded into RAM of computer in an organized manner.

This memory layout is organized in following fashion:-

### 1>Text or Code Segment:-

**Text segment contains machine code of the compiled program. The text segment of an executable object file is often read-only segment that prevents a program from being accidentally modified.**

### 2>Initialized Data Segment:-

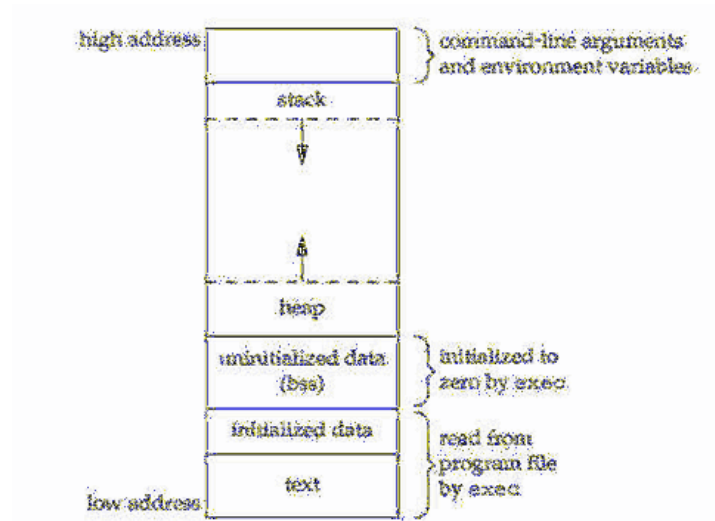
**Initialized data segment, usually called simply the Data Segment.** A data segment is a portion of virtual address space of a program, which contains the global variables and static variables that are initialized by the programmer. **Data segment is not read-only, since the values of the variables can be altered at run time.**

This segment can be further classified into initialized read-only area and initialized read-write area.

```
#include <stdio.h>
```

```
char c[]="rishabh tripathi"; /* global variable stored in Initialized Data Segment in read-write area*/
const char s[]="Hacker Earth"; /* global variable stored in Initialized Data Segment in read-only area*/
```

```
int main()
{
    static int i=11; /* static variable stored in Initialized Data Segment*/
    return 0;
}
```



### 3>Uninitialized Data Segment (bss) :-

**Data in this segment is initialized to arithmetic 0 before the program starts executing.** Uninitialized data starts at the end of the data segment and **contains all global variables and static variables that are initialized to 0 or do not have explicit initialization in source code.**

Uninitialized data segment, often called the “bss” segment, named after an ancient assembler operator that stood for “block started by symbol.”

```
char c;          /* Uninitialized variable stored in bss*/
int main()
{
    static int i; /* Uninitialized static variable stored in bss */
    return 0;
}
```

### 4>Heap :-

Heap is the segment where *dynamic memory allocation* usually takes place. When some more memory need to be allocated using malloc and calloc function, heap grows upward. The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

```
int main()
{
    char *p=(char*)malloc(sizeof(char)); /* memory allocating in heap segment */
    return 0;
}
```

### 5>Stack :-

**Stack segment is used to store all local variables and is used for passing arguments to the functions along with the return address of the instruction which is to be executed after the function call is over.** Local variables have a scope to the block which they are defined in, they are created when control enters into the block. All recursive function calls are added to stack.

The stack and heap are traditionally located at opposite ends of the process's virtual address space

Stack, where automatic variables are stored, along with information that is saved each time a function is called. Each time a function is called, the address of where to return to and certain information about the caller's environment, such as some of the machine registers, are saved on the stack. The newly called function then allocates room on the stack for its automatic and temporary variables. This is how recursive functions in C can work. Each time a recursive function calls itself, a new stack frame is used, so one set of variables doesn't interfere with the variables from another instance of the function.